

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**
**Федеральное государственное образовательное бюджетное учреждение
высшего образования**
«Поволжский государственный университет телекоммуникаций и информатики»

Кафедра «Программной инженерии»
(наименование кафедры)

«УТВЕРЖДАЮ»

Заведующий кафедрой При

наименование кафедры

Макаров И.С.

Фамилия И.О.

подпись,

«21» апреля 2026 г

ЛАБОРАТОРНЫЙ ПРАКТИКУМ

ПО УЧЕБНОЙ ДИСЦИПЛИНЕ

Программно-конфигурируемые сети

(наименование учебной дисциплины)

Для направлений подготовки магистров:

- 27.03.04 «Управление в технических системах»;

(код и наименование направления (специальности) подготовки)

Обсуждено на заседании кафедры
При

«21» апреля 2026 г.

протокол № 9

Самара
2026

СОДЕРЖАНИЕ

Перечень сокращений	4
1. ВВЕДЕНИЕ В ТЕХНОЛОГИЮ SDN	6
1.1. Определение и история развития SDN	6
1.2. Стандартизация в SDN	12
1.3. Область применения SDN	17
1.4. Архитектура программно-конфигурируемой сети	18
1.4.1. Уровень приложений	19
1.4.2. Уровень управления	19
1.4.3. Уровень инфраструктуры	20
1.4.4. Функции конфигурирования оркестратором	20
1.5. Компоненты программно-конфигурируемой сети	20
1.6. Протокол OpenFlow	23
1.6.1. История протокола	23
1.6.2. Версии протокола	23
1.6.3. Уровни сетевого устройства	23
1.6.4. OpenFlow-порты	25
1.6.5. Таблицы OpenFlow	25
1.6.6. Действия (Actions)	27
1.6.7. Сообщения протокола	27
1.7. Алгоритм работы ПКС-сети	29
1.8. Классификаторы протокола OpenFlow	32
1.8.1. Заголовок классификаторов	33
1.8.2. OXM fields	33
1.8.3. OXM classes – reserved	34
2. ЛАБОРАТОРНЫЕ РАБОТЫ	38
Используемые инструменты	38
Лабораторная работа 1. ЗАПУСК МОДЕЛЬНОЙ СЕТИ SDN	40
Лабораторная работа 2. АНАЛИЗ ПАКЕТОВ OpenFlow	44
Лабораторная работа 3. СОЗДАНИЕ ПРАВИЛ ОБРАБОТКИ ПОТОКА НА КОНТРОЛЛЕРЕ SDN	45
Лабораторная работа 4. СОЗДАНИЕ ТОПОЛОГИЙ ПКС	46

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ

API	– Application Programming Interface (интерфейс программирования приложений)
BSS	– Business Support System (система поддержки бизнеса)
CPE	– Customer Premise Equipment (оборудование на стороне пользователя)
CPU	– Central Processing Unit (центральный процессор)
CSP	– Communication Service Provider (провайдер телекоммуникационных услуг)
DHCP	– Dynamic Host Configuration Protocol (протокол динамической маршрутизации)
E2E	– end-to-end (из конца в конец, сквозная услуга)
EMS	– Element Management System (система управления элементами)
ETSI	– European Telecommunications Standardization Institute (Европейский институт стандартизации в области телекоммуникаций)
GRE	– Generic Routing Encapsulation (общая инкапсуляция маршрутов)
IETF	– Internet Engineering Task Force (Инженерный совет Интернета)
IP	– Internet Protocol
IRTF	– Internet Research Task Force (исследовательская группа интернет-технологий)
LTE	– Long Term Evolution
MPLS	– Multiprotocol Label Switching (многопротокольная коммутация по меткам)
NAT	– Network Address Translation (преобразование сетевых адресов)
NF	– Network Function (сетевая функция)
NFV	– Network Functions Virtualization (виртуализация сетевых функций)
NFVI	– Network Functions Virtualisation Infrastructure (инфраструктура виртуализации сетевых функций)
NFVI-PoP	– Network Functions Virtualisation Infrastructure Point of Presence (точка присутствия инфраструктуры виртуализации сетевых функций)
NIC	– Network Interface Card (сетевая карта)
NMS	– Network Management System (система управления сетью)
ONF	– Open Networking Foundation
OSS	– Operation Support System (система поддержки эксплуатации)
QoS	– Quality of Service (качество обслуживания)
REST	– Representational State Transfer (передача репрезентативного состояния)
SDN	– Software Defined Networking (программно-конфигурируемая сеть)
VLAN	– Virtual Local Area Network (виртуальная локальная сеть)
VNF	– Virtual Network Function (виртуальная сетевая функция)

VoIP – Voice over IP

VXLAN– Virtual Extensible LAN

МСЭ-Т – Международный союз электросвязи, сектор стандартизации телекоммуникаций МСЭ

ПКС – программно-конфигурируемая сеть

ПО – программное обеспечение

ЦОД – центр обработки данных

ЧНН – час наибольшей нагрузки



1. ВВЕДЕНИЕ В ТЕХНОЛОГИЮ SDN

1.1. Определение и история развития SDN

В современном мире, при разнообразии поставщиков телекоммуникационного оборудования, прогресс, модификация и сложность различных технологий стремятся к росту количества устройств и объемов передаваемых данных. Классическая архитектура сети, основы которой закладывались в конце 60-х гг. прошлого века, в связи с быстрым ростом многообразия, сложности и важности решаемых задач, устарела и не всегда способна эффективно реагировать на новые потребности рынка. Чтобы полностью преобразовать исторически устоявшуюся архитектуру, необходимо затратить немалые средства, а модернизация отдельных элементов сети не может длиться бесконечно.

Сегодня инфокоммуникационные сети, несмотря на повсеместное использование и масштабное развитие, встречаются на своем пути множество проблем:

- рост количества пользователей и трафика, приводящий к перегрузке сетевых устройств. На сегодняшний день пользователь глобальной сети генерирует больше трафика, чем вся всемирная паутина 30 лет назад. В 2014 г. интернет-трафик вырос по сравнению с 1984 г. в 2,7 млрд раз (Cisco);
- огромное количество протоколов (более 700) и их стеков, число которых с каждым днем только увеличивается;
- устаревшие протоколы, особенно протоколы транспортного и сетевого уровней;
- традиционные сети, которые проприетарны, ограничены для исследований и практически любых изменений. Интерфейс настройки сетевого оборудования иногда зависит даже от модели у одного и того же производителя, следовательно, специалисты в области сетевых технологий должны уметь анализировать системы разных вендоров и работать с большим количеством протоколов;
- оборудование разных производителей, которое довольно часто может конфликтовать между собой, несмотря на универсальность сетевых протоколов;
- настройка сети с большим количеством узлов, требующая много времени, а управление такими сетями осуществляется путем конфигурирования их элементов через специализированные интерфейсы;
- освоение системы оборудования конкретного производителя, требующее переподготовки специалистов [1].

Таким образом, большое количество проблемных факторов ведет к усложнению сетевого оборудования и структуры самих информационных се-

тей и, как следствие, удорожанию сетевого оборудования. В качестве рецепта устранения некоторых сетевых проблем ряд экспертов называют переход к архитектуре программно-конфигурируемых сетей, которые позволяют перевести сетевые элементы под контроль настраиваемого ПО (контроллера). Идеология SDN позволяет упростить сетевые устройства до предела, сделать их «слабыми», а значит, и дешевыми.

Программно-конфигурируемые сети (SDN от англ. Software Defined Networks) – это новая технология построения архитектуры компьютерных сетей, в основе которой лежит перенос функций управления (маршрутизаторов, коммутаторов) в отдельные программные приложения, которые функционируют на отдельном сервере. Таким образом, производится физическое разделение уровней управления и передачи данных. Компоненты данной сети разделены на два класса – коммутаторы и контроллеры. Проприетарные традиционные коммутаторы не позволяют изменять и вносить желаемую функциональность. SDN дает такую возможность. Вся вычислительная нагрузка сосредоточена на контроллерах, которые решают задачи, связанные с прокладкой маршрутов и управлением передачи данных, тем самым разгружаются все остальные сетевые устройства, оставляя им только функцию пересылки трафика. Интерфейсы общения между всеми центральными серверами и контроллерами – открытые, что позволяет попробовать свои идеи и разработки. Допустимо самостоятельно писать приложения для контроллера, это дает возможность использовать некоторые функции, которые ранее были доступны только в качестве аппаратных решений.

На рис. 1 ETSI представила упрощенную хронологию развития технологии SDN.

Сложно представить общую картину истории развития технологии программно-конфигурируемых сетей, если учитывать большое количество стартапов, «поглощений» и инвестиций на мировом рынке SDN, поэтому лучше будет поэтапно рассмотреть самые яркие события на пути становления концепции.

1993–2011

Сама концепция новой сетевой архитектуры, которая позднее получила название SDN, появилась еще в 1993 г., но возможность успешно ее реализовать появилась только в 2007 г., когда американскими преподавателями (Мartiном Касадом, Никоm МакКеоном, Скоттом Шенкером) был разработан новый открытый стандарт общения компьютерных сетей – OpenFlow. Они понимали, что современная сетевая архитектура устарела и, скорей всего, в ближайшем будущем не сможет обеспечить требуемого качества обслуживания.



Рис. 1. Хронология развития технологии SDN (ETSI)

К сожалению, принципиальные изменения в сетевом оборудовании и протоколах – весьма трудное дело и невозможно без привлечения производителя. Переход с оборудования одного вендора на оборудование другого создает большие проблемы и требует немалых затрат ресурсов. В этой ситуации удачной была идея преподавателей разделить уровни управления и передачи данных. Проблему зависимости от оборудования одного производителя Касадо предложил решить с помощью нового сетевого протокола с открытым кодом, который позволяет уйти от «ручного» управления сетью (OpenFlow).

Научное сообщество благоприятно приняло предложенную концепцию. Позже был основан исследовательский центр Open Networking Research Center. Программное обеспечение и все исследовательские программы в рамках ПКС-подхода являются открытыми. Параллельно с этим Касадо, МакКиннон и Шенкер создают коммерческий проект Nicira, первый успешный коммерческий проект в области ПКС, который занимался созданием собственной платформы в сфере виртуализации сетей – Network Virtualization Platform (NVP). Стартап был продан в июле 2012 г. за рекордные 1,26 млрд долл. США при венчурных инвестициях в 50 млн долл. США, его можно считать точкой отсчета для глобальной ПКС-индустрии. После сделки с Nicira на рынке ожидалось, что следующим стартапом, который купит один из гигантов ИТ-индустрии, будет BigSwitch Networks. Компания, выпустившая контроллер Floodlight и средство тестирования OFTest, была основана экс-профессором Стэнфорда Гвидо Аппензеллером и бывшим сотрудником Cisco Кайлом Форстером. Основанный в 2010 г. стартап за 2,5 года собрал почти 50 млн долл. США венчурных инвестиций. Инвесторы полагают, что решения BigSwitch способны изменить образ современных центров обработки данных и того, как в них происходит процесс сетевых коммуникаций. Озвученные выше стартапы являются далеко не единственными, кто занимался разработкой и исследованиями в этой тематике. Множество компаний (Vyatta, Pluribus Networks, Plexxi, LineRate Systems, Contrail Systems) внесли свои решения в разработку платформ и программного обеспечения программно-конфигурируемых сетей. Весной 2011 г. был сформирован консорциум Open Networking Foundation (ONF) в целях развития технологий SDN в целом и протокола OpenFlow в частности. Сегодня членами ONF являются практически все основные поставщики сетевого оборудования (более 130 членов).

Январь 2012

Технологической новинкой заинтересовалась одна из ведущих ISP (Internet Service Provider) компаний мира – Google. Компания начала тестировать коды OpenFlow в 2009 г. еще до выхода официальной версии

протокола. Для того чтобы быть глобальной пользовательской базой с высокой скоростью и доступностью, большие объемы данных должны быть перемещены из одного региона в другой. Эту задачу и должна была решить интеграция сетей SDN в инфраструктуру сети Google. Компания разработала свой собственный высокопроизводительный коммутатор (10 Гбит/с) и открытый стек маршрутизации с поддержкой протокола OpenFlow 1.0. В 2010 г. Google, объединив двенадцать центров обработки данных (G-scale сеть), начала внедрение технологии программно-коммутируемых сетей и на сегодняшний день полностью перестроила свою глобальную сеть, адаптировав ее под OpenFlow. Благодаря этому была улучшена управляемость, производительность, использование и рентабельность в глобальной сети.

Апрель 2013



Linux Foundation – открытый консорциум, участником которого может быть любой человек или любая компания. Организация является гидом для компаний, которые хотят строить свои решения на базе OpenSource, и предоставляет пространство для обсуждения насущных вопросов техническим сообществам, разработчикам приложений, промышленным заказчикам и конечным пользователям. В апреле 2013 г. консорциум начал работу над новым проектом OpenDaylight, в рамках которого многие ведущие производители отрасли объединили свои усилия в целях создания единой открытой платформы для организации работы программно-конфигурируемых сетей. В итоге совместной работы был подготовлен открытый фреймворк, на базе которого заинтересованные компании и отдельные разработчики могут создавать готовые продукты и сервисы для SDN-сетей. Например, SDN очень востребованы в облачных системах, в которых виртуализация сетевого уровня позволяет динамически предоставлять клиентам желаемые виртуальные сетевые ресурсы. Используя уже готовый унифицированный фреймворк, поддерживаемый всеми производителями оборудования, появится возможность существенно упростить и ускорить создание конечных приложений и сервисов для SDN, основанных на уже сформированном едином высокоуровневом стеке, не привязанном к решениям отдельного вендора. Первый выпуск OpenDaylight кода под названием «Hydrogen» включает реализацию контроллера для SDN-сети, систему для создания виртуальных сетей, набор плагинов для поддержки различных протоколов и набор улучшений для виртуальных коммутаторов.

Таким образом, OpenDaylight выступает в роли полнофункциональной SDN-платформы, пригодной для прямого развертывания в различных сетевых окружениях, без необходимости использования каких-либо дополнительных компонентов. Расширения и дополнительные возможности по-

ставляются в форме плагинов. Названия релизов соответствуют элементам таблицы Менделеева: в феврале 2013 г. вышел Hydrogen (водород), в сентябре 2014 г. – Helium (гелий), третий релиз вышел в 2015 г. под названием Lithium (литий).

Ноябрь 2013

Программа Supplier Domain 2.0, которую AT&T ввела в ноябре 2013 г., является релизом начальной версии Domain 1.0. Основной стратегией является переход крупнейших операторов связи и телекоммуникационных компаний к крупномасштабному использованию SDN и виртуализации сетевых функций (Network Function Virtualization – NFV). Задача компаний к 2020 г. – трансформировать 75 % своей сетевой архитектуры под архитектуру SDN. К концу 2014 г. 40 % приложений AT&T уже было переведено на новую структуру сети, что позволило на 50 % увеличить эффективность пропускной способности по сравнению с традиционной архитектурой

Март 2015

Open Network Operating System (ONOS) – это проект, который был основан в ноябре 2014 г. некоммерческой группой Open Networking Lab (ON.Lab), которая в свою очередь состояла из пионеров в разработке и стандартизации технологии SDN. Проект занимается разработкой открытого исходного кода для SDN и виртуализации сетевых функций. Платформа имеет отличную архитектуру, ориентированную на операторские сети и сервисные требования, что делает ее высокопроизводительной, надежной и безопасной. В марте 2015 г. ONOS совместно с компанией Huawei и фондом ONF объявила о создании открытой инновационной экосистемы типа SDN (приставка «эко» не связана с окружающей обстановкой). Целью сотрудничества было поставлено создание единой, открытой и программируемой сетевой архитектуры SDN, которая будет отвечать соответствующим потребностям операторов связи и принесет наибольшую выгоду. Сегодня ONOS является частью Linux Foundation.

Прогноз к 2020 г.

Осознав все преимущества, мобильные операторы и операторы широкополосного доступа начали делать инвестиции в технологию программно-конфигурируемых сетей, это позволяет увидеть детальный прогноз развития и сделать вывод о дальнейшем положении SND на мировом рынке. Согласно отчету агентства Research and Markets, инвестиции сервис-провайдеров в программно-конфигурируемые сети и виртуализацию сетевых функций к концу 2020 г. составят более 20 млн долл. США, а среднегодовой темп роста инвестиций в 2015–2020 гг. составит 54 %.

1.2. Стандартизация в SDN

Технология SDN привлекла большое внимание не только со стороны поставщиков, но и со стороны бизнеса. Вследствие этого, все большее количество компаний начинают проявлять интерес к использованию ПКС-сетей, что требует разработки системы однородных стандартов для открытого применения. На сегодняшний день существуют несколько некоммерческих организаций, рабочих групп и сообществ, которые предлагают свои определения и стандарты в этой области. Отметим некоторые из них.

- **Open Networking Foundation (ONF**, фонд открытых сетевых технологий) – одна из первых некоммерческих организаций, целями которой было продвижение и стандартизация принципиально нового подхода к передаче данных.

Определение программно-конфигурируемых сетей, предложенное консорциумом ONF: *«Динамичная, управляемая и адаптируемая сетевая архитектура, в которой разделены уровни управления сетью и передачи данных, что позволяет обеспечить программное управление сетью и абстрагирование/изоляция (уровня) сетевой инфраструктуры от (уровня) приложений и сетевых услуг/сервисов».*

Основной задачей ONF является продвижение стандарта OpenFlow, который позволяет осуществлять удаленное управление уровнем передачи данных. Стандарт OpenFlow – первый стандарт SDN. В настоящее время в ONF продолжается работа по анализу требований к SDN, развитию стандарта OpenFlow в соответствии с запросами, возникающими при коммерческом развертывании SDN, а кроме того, создаются новые стандарты в целях расширения возможностей SDN.

Структура ONF состоит из технических сообществ, которые разделены на области, советы и группы. Каждая из областей включает в себе определенный набор рабочих групп, которые отвечают на конкретные вопросы, связанные с программно-конфигурируемыми сетями (концепция SDN, архитектура, сертификация, программные платформы, программное обеспечение), и сотрудничают с мировыми экспертами в области разработки стандартов SDN и OpenFlow. Советы разрабатывают стратегию, контролируют операционное исполнение и технические направления организации. Группы работают помощниками в обеспечении бесперебойной работы консорциума, руководят и консультируют ONF для достижения общих целей организации.

- **Internet Engineering Task Force (IETF**, группа по решению задач проектирования Интернета) – открытое международное сообщество сетевых проектировщиков, операторов и провайдеров, которые заинтересованы исследованиями и эволюцией сети Интернет.

Определение программно-конфигурируемых сетей, предложенное группой IETF: *«SDN – подход к построению сетей, обеспечивающий прямое управление ресурсами и сетями, а также их распределение за счет добавления собственных средств обработки, администрирования и программного управления посредством открытых сетевых интерфейсов и абстракции (абстрагирования, изоляции) уровня сети».*

Архитектура функционирования сообщества состоит из нескольких рабочих групп, действия которых распределены согласно основным выделенным тематикам, таким как маршрутизация, транспорт, безопасность и т. д.

Разработка аналитических и стандартизирующих документов IETF, относящихся к SDN, стартовала в конце 2012 г. и сейчас находится на начальных этапах. Следует отметить, что ONF и IETF определяют два совершенно разных подхода к реализации SDN. ONF определяет OpenFlow как интерфейс между двумя уровнями и считает, что весь потенциал программно-конфигурируемых сетей можно раскрыть благодаря протоколу, который осуществляет перенос данных с одного уровня на другой. IETF предлагает свой элемент архитектуры сети SDN – PCE (Path Computation Element), задача которого вычислять маршруты LSP (Label Switch Path). Элемент может быть реализован как роутер, часть OSS-системы или как виртуальный элемент, поднятый в облачной среде. В целом принцип работы подхода IETF выглядит так: сетевой элемент при прокладке маршрута обращается к PCE, тот, зная всю сетевую топологию и статус сети, вычисляет оптимальный маршрут. Затем PCE возвращает узлу данные для LSP, которые передаются по сети с помощью сигнального протокола.

Исследования по SDN в IETF осуществляет исследовательская группа Software-Defined Networking Research Group (SDNRG) – группа по SDN, сформированная в составе исследовательской группы интернет-технологий IRTF. Целью SDNRG является изучение различных аспектов SDN для определения, распространения и применения подходов к реализации концепции в краткосрочной перспективе, а также выявление вопросов, требующих дальнейшего изучения. В частности, целевыми вопросами для РГ являются масштабируемость, абстракции, языки и парадигмы программирования, полезные при реализации SDN. В настоящий момент SDNRG представила документы, в которых рассматриваются вопросы сокращения объемов контрольной информации в сети SDN, уровни SDN и терминология архитектуры SDN, концепция SDN с точки зрения поставщика услуг, анализ кластеров контроллеров SDN в крупномасштабных сетях.

• **European Telecommunications Standards Institute (ETSI**, Европейский институт по стандартизации в области телекоммуникаций) – некоммерческая организация по разработке стандартов в области информационных и коммуникационных технологий, официально признанная Европейским

союзом. Институт разрабатывает стандарты фиксированной, подвижной, радио, конвергентной связи, а также телевидения и интернет-технологий. В составе ETSI была выделена группа отраслевой спецификации (Industry Specification Group, ISG) по разработке концепции виртуализации сетевых функций (служб) (Network Functions Virtualisation, NFV). Концепция NFV предполагает замещение разнообразных сетевых устройств стандартизированными высокопроизводительными серверами, коммутаторами и системами хранения данных с реализацией сетевых функций (служб) программным обеспечением.

Предполагается, что ISGNFV разработает требования и архитектуру NFV, рассмотрит вопросы управления, оркестровки служб, архитектуры ПО, производительности и переносимости, надежности и устойчивости, безопасности и миграции к NFV. Концепция NFV сходна с концепцией SDN, однако на текущем, начальном этапе работы степень их взаимного соотношения не является точно определенной. Согласно ETSI, концепция NFV в большой степени дополняет SDN, но концепции независимы, каждая из них может быть реализована отдельно. Некоторые считают, что совместное использование технологий открывает широкие возможности для обеспечения связности инфраструктуры. Использование их создаст беспрецедентные условия для быстрого развертывания отказоустойчивой, ориентированной на сетевые приложения архитектуры с высоким уровнем безопасности и возможностью обеспечения гарантированного качества услуг. В январе 2015 г. ISG представила 11 спецификаций, завершив, таким образом, первый этап работы.

• **International Telecommunication Union** (ITU, Международный союз электросвязи) – специализированное учреждение Организации Объединенных Наций в области информационно-коммуникационных технологий. Организация занимается распределением радиочастотного спектра и спутниковых орбит в глобальном масштабе, разрабатывает технические стандарты на международном уровне, обеспечивающие возможность эффективного присоединения сетей и технологий, и стремится улучшить доступ к ИКТ для недостаточно обслуживаемых сообществ всего мира. МСЭ-Т занимается исследованием технических, эксплуатационных, тарифных и других вопросов, выпускает рекомендации в целях стандартизации электросвязи на международном уровне. В ходе Всемирной ассамблеи МСЭ по стандартизации электросвязи в Дубае в 2012 г., где обсуждались вопросы SDN, было высказано согласованное мнение, что программно-коммутируемые сети коренным образом преобразуют отрасль электросвязи и ИКТ в ближайшие десятилетия, обеспечат отрасли многочисленные преимущества. В условиях быстро растущего интереса к SDN со стороны значительного количества компаний необходима система стандартов для широкого

применения SDN. По результатам обсуждения тематики SDN была принята Резолюция 77 «Работа по стандартизации в области организации сетей с программируемыми параметрами в Секторе стандартизации электросвязи МСЭ», поручившая Исследовательской комиссии (ИК) 13 в следующем исследовательском периоде расширить и ускорить работы в области разработки архитектуры и требований к SDN, а также представить рекомендации Консультативной группе по стандартизации электросвязи (КГСЭ) относительно порядка рассмотрения вопросов, выходящих за рамки мандата ИК 13. КГСЭ поручено изучить проблему, рассмотреть вклады ИК 13 и других ИК и принять необходимые меры для активизации деятельности по стандартизации SDN в МСЭ-Т.

Определение программно-конфигурируемых сетей, предложенное Международным союзом электросвязи: *«SDN технология построения сетей, которая позволяет реализовать централизованный, программируемый уровень управления и изоляцию (абстракцию) уровня данных, при этом уровни управления и данных разделены, благодаря чему операторы сетей связи могут напрямую управлять своими виртуальными ресурсами и сетями».*

В настоящее время исследованиями в области SDN в МСЭ-Т занимаются ИК 13 (архитектуры и функциональные требования к SDN) и ИК 11 (эталонные архитектуры сигнализации SDN, требования к сигнализации и протоколам SDN, включая протоколы взаимодействия, а также тестирование на соответствие и взаимодействие). Интерес к SDN проявляют ИК 15 (транспорт в SDN) и ИК 17 (безопасность в SDN).

Существует некоторое количество представителей, которые акцентируют свое внимание на решении частных вопросов построения и эксплуатации ПКС-сетей.

Рассмотрим отдельные сообщества:

– **Optical Internetworking Forum** (OIF, оптический межсетевой форум) – некоммерческая организация, основанная в 1998 г. Участники форума разрабатывают соглашения по реализации для оборудования оптических сетей, а также занимаются демонстрацией, тестированием и построением требований к SDN со стороны операторов и поставщиков услуг в части транспортных сетей, проводят исследования на соотносимость архитектуры SDN с архитектурой оптических сетей с автоматической коммутацией;

– **Broadband Forum** (BBF, форум широкополосных сетей) – некоммерческая организация, сосредоточенная на разработке спецификаций для «умных» и высокоскоростных широкополосных сетей, спектр которых простирается от DSL до технологий создания максимально эффективных и управляемых широкополосных сетей, объединяющих операторское и абонентское оборудование в единую IP-платформу предоставления услуг.

В рамках рабочей группы «Инновационные услуги и рыночные требования» основан проект SD-113 (коммерческие требования и структура SDN в широкополосных телекоммуникационных сетях), на основе которого предполагается проведение исследований по поиску вариантов перехода к сетям SDN, включая способы внедрения функциональности и поддержки ПКС-сетей оборудованием при обновлении программного обеспечения.

Качественное решение проблем интеграции и стандартизации любой технологии невозможно без проведения испытаний разработанных решений в области исследования данной тематики. На рис. 2 представлено общее распределение стандартизирующих организаций по направлениям разработки архитектуры SDN. Испытанием разрабатываемых стандартов и разработкой открытых решений SDN занимаются несколько организаций:

1) OpenDaylight – объединение отраслевых производителей, включая IBM, Juniper Networks, Cisco, Red Hat, VMware, Citrix, Ericsson, Microsoft, NEC, Big Switch Networks, Brocade Communications Systems. Целью объединения является создание единой открытой платформы SDN, разработка открытой структуры классов (фреймворка) в качестве основы для создания готовых продуктов и сервисов различными участниками рынка;

2) Open vSwitch – проект по разработке программного коммутатора SDN с открытым исходным кодом для применения в виртуализированной сетевой среде. В проекте участвуют компании Citrix, Red Hat, Canonical, Oracle, FreeBSD Foundation, Nicira;

3) OpenStack – проект по разработке комплекса бесплатных открытых программных средств для создания облачной вычислительной инфраструктуры и хранилищ данных, включающий поддержку SDN. В проекте представлены компании AT&T, AMD, Brocade Communications Systems, Canonical, Cisco, Dell, EMC, Ericsson, Groupe Bull, HP, IBM, Inktank, Intel, NEC, Rackspace Hosting, Red Hat, SUSE Linux, Yahoo!, VMware.

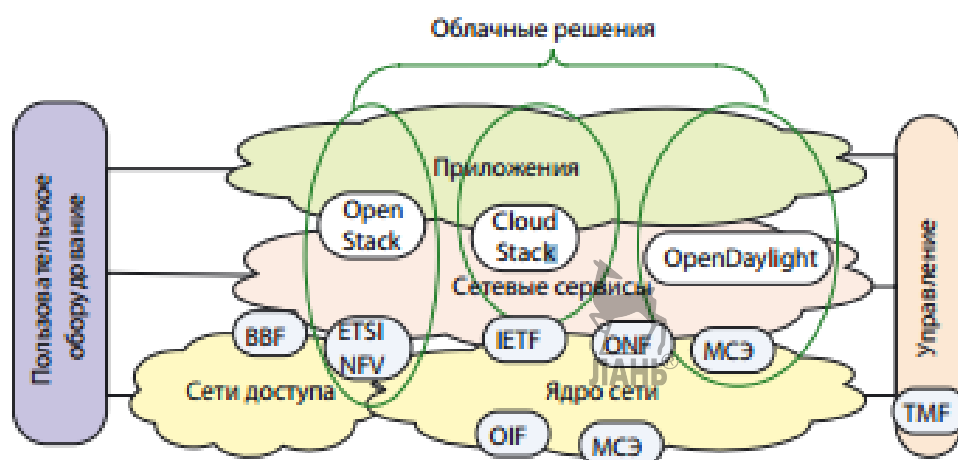


Рис. 2. Распределение стандартизирующих организаций по направлениям разработки архитектуры SDN

1.3. Область применения SDN

SDN применяются в коммутации, виртуализации облачных приложений и виртуализации средств безопасности сетевых решений. Основные направления, где программно-коммутируемые сети пытаются найти свое место, следующие:

- 1) сети ЦОД;
- 2) облачные технологии;
- 3) сети провайдеров;
- 4) корпоративные сети;
- 5) локальные сети (домашние);
- 6) средства обеспечения безопасности.

В локальных сетях модернизация будет довольно нескоро, но разработчики о такой возможности говорят. Если модернизация произойдет, то SDN можно будет применять для настройки локальных сетей с управлением трафиком внутри них, кроме того, операторы смогут удаленно настраивать такие сети.

К тому же архитектура SDN работает в реальной сети сотовой связи и может использоваться в mesh-сетях.

Основные преимущества внедрения программно-коммутируемых сетей в компаниях со сложной ИТ-инфраструктурой:

1) снижение стоимости эксплуатации компьютерных сетей и, как следствие, увеличение прибыли за счет снижения расходов на управление;

2) повышение производительности. В связи с тем, что с коммутаторов снимаются нагрузки по обработке линии управления, программно-коммутируемая сеть дает возможность этим устройствам направить все свои ресурсы на ускорение перемещения трафика;

3) ускорение реализации и тестирования новых сервисов. Программные средства программно-коммутируемых сетей позволяют администраторам добавлять новые функции к уже имеющейся сетевой архитектуре;

4) упрощение процесса администрирования. На централизованном контроллере программно-коммутируемой сети системный администратор может наблюдать сеть в едином представлении, что способствует повышению удобства управления, обеспечения безопасности и выполнения других задач;

5) повышение безопасности. Программно-коммутируемая сеть предоставляет возможность администратору четко видеть все потоки трафика, поэтому он будет гораздо легче замечать вторжения, определять приоритеты различных типов трафика, разрабатывать правила реагирования сети при заторах и проблемах с оборудованием;

6) применение облачных технологий. В облаках данные и приложения размещены на компьютерах, которые взаимодействуют по сети, и данная технология способна дать требуемый облакам уровень «интеллектуальности» сетей.

1.4. Архитектура программно-конфигурируемой сети

Существуют две независимые модели развития архитектуры SDN. Одна из них продвигается консорциумом ONF, другая – IETF. Кроме этого, некоторые открытые сообщества пытаются разработать протоколы взаимодействия между NFV и SDN. Например, группой ETSI NFV ISG представлена модель MANO (Management And Orchestration), а консорциумом MEF (Metro Ethernet Forum) была разработана концепция LSO (Lifecycle Service Orchestration), которая является дополнением модели MANO и неким связующим звеном между вышеописанными архитектурами (ONF SDN и ETSI NFV MANO). Наибольшую популярность и экономические показатели имеет модель ONF на базе протокола OpenFlow. Консорциум продвигает трехуровневую модель, состоящую из уровней инфраструктуры, управления и приложений, если подробно рассмотреть информационные потоки в архитектуре SDN (рис. 3), то можно увидеть два направления обмена информацией. Первый поток – между бизнес-приложениями конечных пользователей и уровнем управления, второй – между уровнем физических сетевых устройств и опять же уровнем управления. Первый поток получил название «северный интерфейс» (northbound API), а второй – «южный интерфейс» (southbound API). В качестве «северного интерфейса» выступает протокол на основе REST API (REST – общие принципы организации взаимодействия приложения с сервером, API – интерфейс программирования приложений, состоящий из набора готовых кодов, предоставляемых приложением для использования во внешних программных продуктах), а в качестве «южного интерфейса» – протокол OpenFlow.

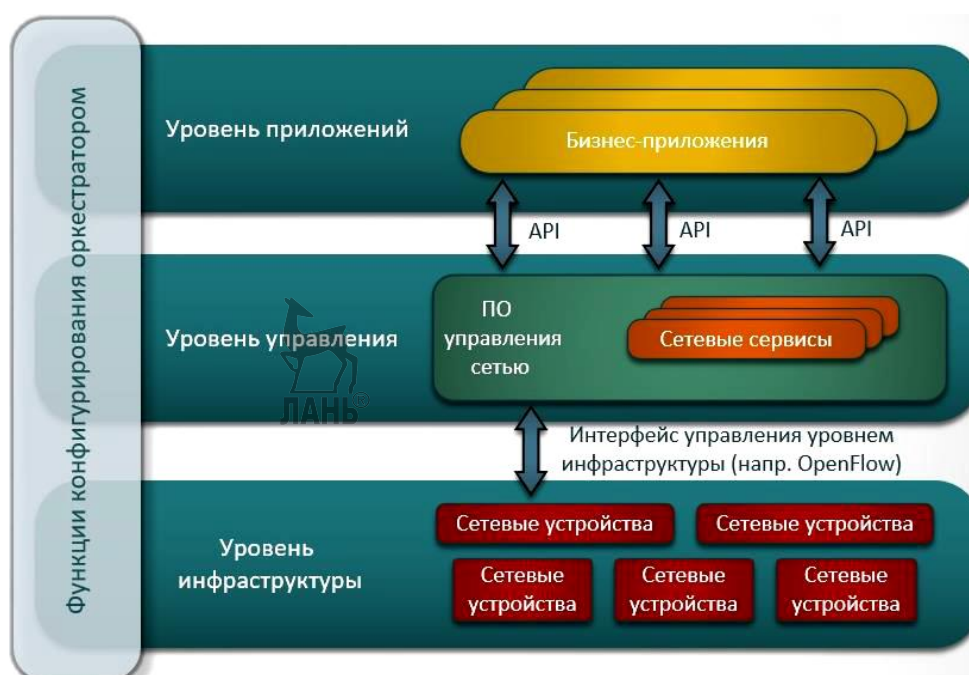


Рис. 3. Трехуровневая модель SDN консорциума ONF

Концепция обладает свойством изолированности функций управления от функций форвардинга данных (disaggregation), в связи с этим каждый из уровней модели может быть рассмотрен по отдельности, а не в качестве единой интегрированной системы.

1.4.1. Уровень приложений

Совокупность приложений, которые напрямую взаимодействуют с SDN-контроллером, запрашивая необходимые ресурсы посредством API, решает высокоуровневые задачи по управлению сетью. Примером таких приложений могут служить средства мониторинга, аналитики или бизнес-приложения. Например, конкретное бизнес-приложение Microsoft Lync и его основная роль – изменение сети в режиме реального времени под текущие нужды обслуживаемой программы (изменение Quality of Service или создание VPN тоннеля между двумя абонентами).

1.4.2. Уровень управления

Уровень управления осуществляет низкоуровневое логически централизованное управление, которое обеспечивает форвардинг трафика на инфраструктурном уровне с помощью открытого интерфейса OpenFlow. Уровень управления постоянно осуществляет мониторинг всей сети и способен управлять всеми сетевыми устройствами. Поэтому приложениям и сетевым политикам сеть представляется как единый логический коммутатор (рис. 4).

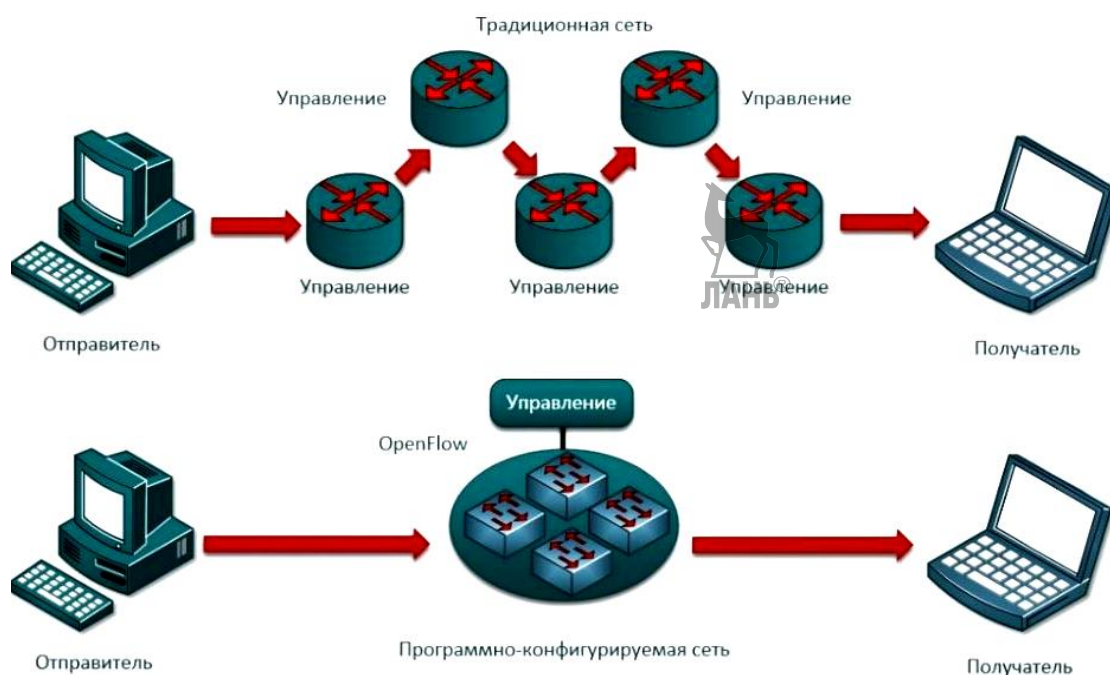


Рис. 4. Управление сетью в традиционной сети и сети SDN

1.4.3. Уровень инфраструктуры

Состоит из среды передачи данных и коммутаторов SDN (далее OpenFlow-коммутаторы), которые могут быть как логическими, так и физическими элементами сети.

1.4.4. Функции конфигурирования оркестратором

Вертикальная сквозная в модели представлена не как уровень архитектуры SDN, а как среда определения и контроля уровней управления и инфраструктуры.



1.5. Компоненты программно-конфигурируемой сети

На протяжении многих лет традиционные сети стремились уйти от централизации. Сегодня они являются децентрализованными, каждое устройство изучает сеть вокруг себя самостоятельно. Ключевая особенность традиционной сети как управляемой структуры – распределенность. Существующая парадигма ПКС-сетей повторно приводит к истокам развития, а точнее к идее централизованного контроля. Рассмотрим второй подход реализации SDN-сетей на базе специальных аппаратных коммутаторов, которые взаимодействуют с контроллерами по протоколу OpenFlow.

Рассмотрим кратко компоненты архитектуры сети.

1. Оркестратор – представленная в виде аппаратного устройства или программного обеспечения платформа автоматизации, которая используется приложениями для конфигурирования устройств уровней управления и инфраструктуры (например, корректирует работу нескольких контроллеров) и выполняет конкретные сервисные запросы. Существует несколько моделей взаимодействия контроллеров и оркестратора.

2. Контроллер – вычислительное устройство (например, сервер), на котором функционирует специализированная платформа, состоящая из сетевых управляющих приложений и сетевой операционной системы (COC):

- COC – готовый фреймворк, задача которого предоставить интерфейс прикладного программирования для сетевых приложений по контролю и управлению сетью целиком, а также реализация механизмов управления таблицами одного или нескольких OpenFlow-коммутаторов (добавление, удаление, модификация правил, сбор статистики). Разработано более 30 сетевых операционных систем для контроллеров SDN-сетей: Ryu, Maestro, MUL, Beacon, Floodlight, POX, NOX, In-kernel и т. д.;

- SDN-приложение – программная реализация различных сетевых функций и сервисов (маршрутизация, балансировка нагрузки, фильтрация тра-

фика, шлюзы, сетевые экраны, шифрование, DPI, NAT, DHCP, DNS и т. д.). На данный момент реализованы целые онлайн-магазины SDN-приложений, которые позволяют загружать приложения на контроллер и сразу же их использовать.

Архитектура контроллера многоуровневая. Требования к структуре пока не стандартизированы, и по сути контроллер является «черным ящиком». На рис. 5 приведена обобщенная возможная схема архитектуры компонентов контроллера.

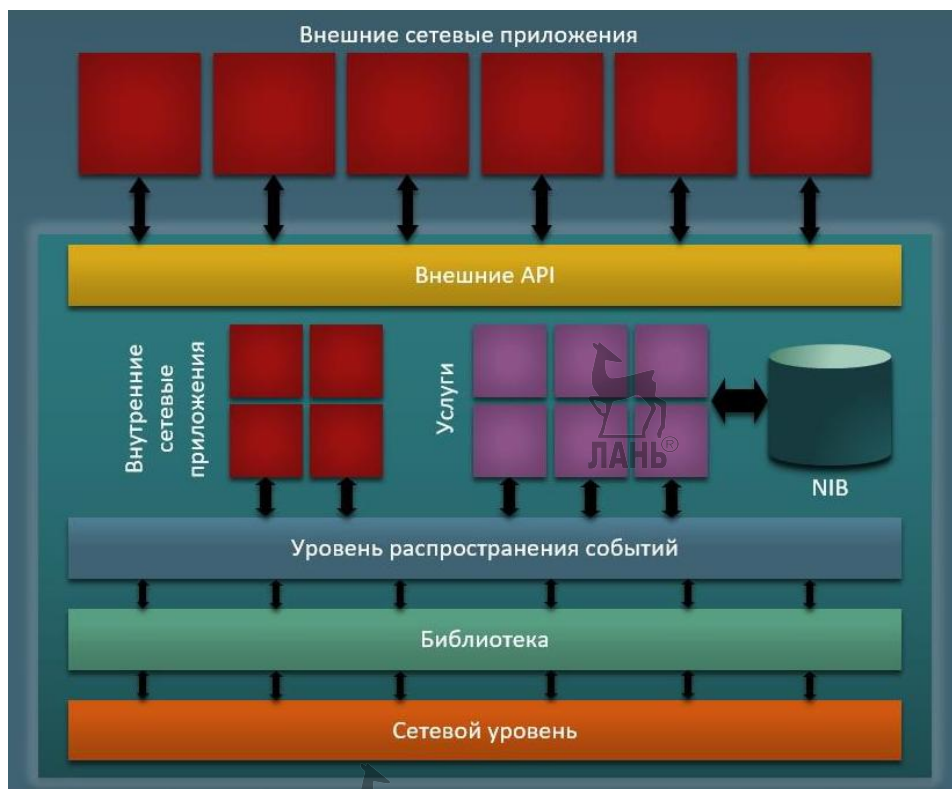


Рис. 5. Обобщенная архитектура SDN-контроллера

Краткий алгоритм работы выглядит так: сначала необходимо достоверно принять пакеты через защищенный канал, далее следует выделить OpenFlow-сообщения, понять, от какого коммутатора они приходят, разобрать, сгенерировать события, на которые подписаны приложения и сервисы контроллера. После этого приложение получает сообщение «packet-in», обрабатывает его и отправляет ответ обратно на уровень взаимодействия с коммутатором. Важно отметить, что приложения можно интегрировать внутрь контроллера или же запускать их с отдельного сервера и «общаться» с контроллером через унифицированный интерфейс. Выбор зависит от требований к производительности системы. Важный компонент структуры контроллера Network Information Base (NIB) – это сетевая информационная база, которая содержит всю информацию о графе сети, т. е. информацию о всех устройствах, которые находятся под управлением данного контрол-

лера. База строится с помощью протокола OpenFlow для полного представления топологии сети и мониторинга за сетевыми узлами в режиме реального времени.

3. OpenFlow-коммутатор – простое программируемое сетевое устройство, выполняющее лишь функции коммутации данных (forwarding) согласно инструкциям контроллера. Основные составляющие OpenFlow-коммутатора, принцип работы и уровни устройства описаны в п. 1.6.3.

4. Программный интерфейс (northbound API) [5] – открытый интерфейс программирования, позволяющий программировать контроллер извне. Предназначен для создания экосистемы приложений. Пользователями данного API являются все разработчики сетевых приложений.

Новая модель программных интерфейсов базируется на нескольких компонентах:

- NETCONF (Network Configuration Protocol) – протокол определяет простой механизм управления сетевым устройством. Описывает процессы манипуляции (извлечение, восстановление, загрузка) с данными конфигурации устройства. Протокол позволяет устройству сформировать программный API-интерфейс, который используется приложениями для отправки и получения частичных или полных наборов данных. NETCONF использует технологию удаленного вызова процедур, реализованную на базе XML (RPC-XML), для запуска функций или процедур в другом адресном пространстве;

- YANG – язык моделирования данных. Модули YANG позволяют описать структуру типов данных в формате XML для протокола NETCONF. Описываемые типы данных: конфигурация устройства, данные о состоянии, удаленный вызов процедур, уведомления. YANG-модули – это иерархическая структура данных, представленная в виде дерева, в которой каждый узел имеет имя, а также значение или набор дочерних узлов. Язык содержит четкие и краткие описания взаимодействия между узлами;

- RESTCONF – чтобы понять, что это за протокол, предварительно немного обобщим вышеописанные понятия. Протокол NETCONF описывает механизм работы четырех базовых функций CRUD (сокр. от англ. create, read, update, delete) над данными конфигурации устройств сети, язык YANG определяет синтаксис и семантику данных для NETCONF, а протокол RESTCONF, работая через HTTP, используется для упрощенного доступа к данным протокола NETCONF. Таким образом, RESTCONF позволяет реализовать CRUD операции без предварительных знаний о протоколе NETCONF.

В консорциуме ONF разработкой и стандартизацией программного интерфейса занимается рабочая группа Architecture and Framework. Группой представлена схема «северного интерфейса».

1.6. Протокол OpenFlow

OpenFlow – протокол взаимодействия между сетевыми устройствами (OpenFlow-коммутаторами) программно-конфигурируемой сети SDN и централизованным контроллером.

1.6.1. История протокола

История протокола берет свое начало раньше концепции SDN и даже раньше самого протокола OpenFlow. В Стэнфордском университете в рамках программы «Дизайн Интернета с чистого листа» («Clean Slate Design for the Internet») Мартином Касадо был разработан проект Ethane. Проект моделирует корпоративную сеть на 400 машин, в которой даже теоретически не могут распространяться вирусы. Машины внутри сети обмениваются информацией, но не свободно, а получая разрешение на обмен трафиком определенного вида. Таким образом, вся сеть находится под управлением единой системы фильтров. Изначально Ethane разрабатывался как подход к борьбе со спамом и вирусами, но идея централизации управления сетевыми устройствами и реализация определенного уровня абстракции сети после закрытия проекта были использованы в дальнейших разработках OpenFlow и SDN. Концепция протокола была сформулирована в 2008 г., в декабре 2009г. появилась первая версия спецификации – OpenFlow 1.0. С 2011 г. продвижением протокола занимается ONF.

1.6.2. Версии протокола

Развитие протокола прошло через ряд версий, на сегодняшний день последняя выпущенная консорциумом ONF версия – это версия 1.5.1. Первая версия протокола является широко используемой, однако имеет проблемы с масштабируемостью и большое количество ограничений. Версии 1.1 и 1.2 считаются переходными, и практически никто из производителей не реализует их поддержку. Наиболее перспективной считается версия 1.3 (включена поддержка MPLS-тэгов, счетчиков, Provider Backbone Bridging (PBB) и еще некоторых полезных функций). Протокол является полностью открытым, однако его разработки контролируются закрытой группой, состоящей примерно из 150 компаний, формирующих ONF. Разработки ведутся в скрытом режиме, и их результат виден только после публикации новой версии в качестве стандарта.

1.6.3. Уровни сетевого устройства

Суть концепции программно-конфигурируемых сетей заключается в выносе плоскости управления на отдельное устройство. Протокол OpenFlow –

это один из множества инструментов реализации разделения двух плоскостей (Control Plane и Data Plane).

Чтобы четко выделить область работы протокола и определить его функции, рассмотрим задачи трех независимых уровней, из которых состоит практически каждое сетевое устройство независимо от концепций и архитектур сетей:

- Management plane – предоставление интерфейса управления и мониторинга, с помощью которого производится конфигурация устройства;
- Control Plane – принятие решений о перенаправлении PDU (Protocol Data Unit) путем воздействия на Data plane. Состоит не только из протоколов маршрутизации (OSPF, IS-IS), но и протоколов, контролирующих взаимодействие между смежными узлами (BFD, STP, LACP);
- Data Plane – пересылка PDU согласно правилам, определенным таблицей, которая может состоять из MAC-адресов и соответствующих исходящих портов. Data plane не отвечает за формирование таблицы FIB (Forwarding Information Base).

Management plane, в концепции SDN, не так плотно взаимодействует с протоколами типа OpenFlow, в отличие от Control и Data plane. Уровень данных представлен OpenFlow-коммутаторами (рис. 6).

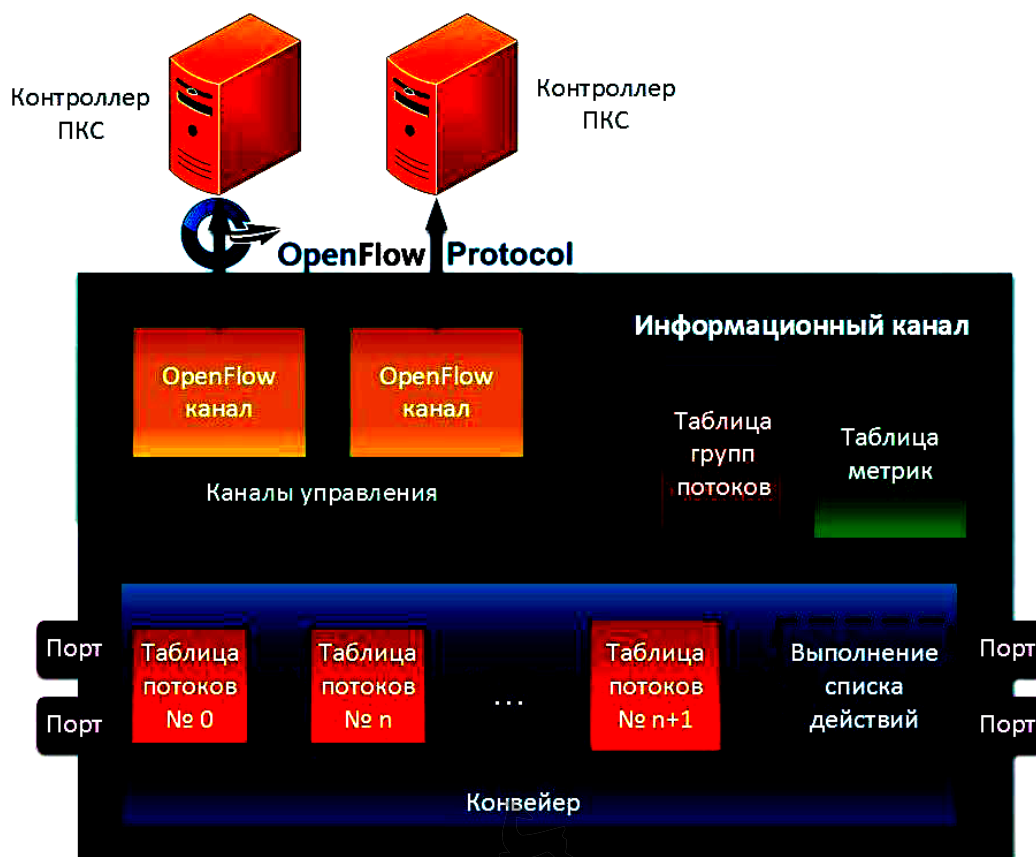


Рис. 6. Основные компоненты OpenFlow-коммутатора

1.6.4. OpenFlow-порты

Ничем не отличаются от портов обычного коммутатора уровня L2, каждый порт имеет входной/выходной буфер и уникальный номер, который выступает в роли имени порта.

Протокол OpenFlow определяет набор стандартных портов:

- физические (physical) – физические (аппаратные) порты OpenFlow-коммутатора;

- логические (logical) – порты, которые не обязательно соответствуют физическим интерфейсам. Представляют собой более высокий уровень абстракции и могут быть определены без использования протокола OpenFlow (например, интерфейс кольцевой проверки (loopback), нулевой или туннельный интерфейс);

- зарезервированные (reserved) – используются для внутренней обработки трафика и для гибридных типов внедрения (OpenFlow + традиционная сеть). Представляют собой специальное действие по пересылке пакета, могут быть опциональными (Local, Normal, Flood) и обязательными:

- All – дублирование пакета на все порты, определяемые действием (используется только для исходящих пакетов);

- Controller – инкапсуляция пришедшего пакета и отправка по OpenFlow-каналу на контроллер (используется как для входящих, так и для исходящих пакетов);

- Table – действие по обработке и передаче пакета в первую таблицу в конвейере;

- In port – отсылка пакета, поступившего на вход;

- Any – специальное действие, используемое в некоторых OpenFlow-командах, когда не указан адрес порта.

1.6.5. Таблицы OpenFlow

Таблица потоков (Flow table)

Каждый OpenFlow-коммутатор содержит одну или несколько уникальных таблиц, которые он заполняет на основе данных, полученных от центрального контроллера. Так как по сети передаются не отдельные пакеты, а потоки данных, то такая таблица получила название Flow table (таблица потоков). Под потоком понимается совокупность пакетов (сеанс связи), определяемая по различным признакам и ограниченная только возможностями реализации самих таблиц (например, TCP-сессия, пакеты с определенного MAC- или IP-адреса, пакеты с одинаковым номером порта OpenFlow-коммутатора).

Составляющие таблицы потоков:

- классификатор – наименование поля заголовка пакета (field) и строка из двоичных символов и символа неопределенности (pattern). Запись о потоке применима к пакету, если все двоичные символы строки pattern поля field совпали с соответствующим полем пришедшего пакета и все классификаторы (список пар (field, pattern)) совместимы с заголовком пакета;
- приоритет – натуральное число из определенного диапазона, указывающее степень значимости записи о потоке в таблице потоков. Используется для избирательного применения записей о потоках к конкретному пришедшему пакету (для обработки выбирается запись с наибольшим приоритетом);
- счетчики – содержат статистические данные о работе записи (например, информацию о том, сколько пакетов было обработано в соответствии с данной записью о потоке);
- инструкции – операция, которая содержит либо множество действий, чтобы добавить их в список действий (action set), либо действия, которые немедленно применяются к пакету или модифицируют процесс обработки пакета на конвейере, в зависимости от этого различают несколько типов инструкций:
 - 1) Apply-Actions – применяются незамедлительно без изменения списка действий. Могут использоваться для модификации пакета между двумя таблицами или для выполнения нескольких действий одного и того же типа;
 - 2) Clear-Actions – удаляет все действия из списка действий;
 - 3) Write-Action – заносит действия в список действий. Если действие данного типа присутствует в списке, то оно перезаписывается, в противном случае добавляется;
 - 4) Write-Metadata metadata/mask – записывает маскированные метаданные в поле метаданных;
 - 5) Goto-Table next-table-id – указывает следующую таблицу в процессе конвейерной обработки;
- Временные метки – метки, которые определяют срок жизни записи в таблице потоков (максимальное время пребывания и максимальный срок простоя правила).

Таблица групп потоков (Group table)

Кроме таблиц потоков в протоколе OpenFlow, начиная с версии 1.1, реализуется механизм OpenFlow-групп. Группа представляет из себя список подмножеств, состоящих из набора действий и связанных параметров. Каждая группа представлена в виде записи в таблице групп потоков. В зависимости от того, какие действия в группе необходимо выполнить для получаемых пакетов, различают несколько типов групп:

- all – выполняются все подмножества действий в группе (multicast);

- **select** – выполняется одно подмножество действий в группе. На основе определенного алгоритма пакеты посылаются в единственное подмножество в группе (распределение нагрузки);
- **indirect** – выполняется одно из определенных подмножеств в группе. Indirect используется для указания общего идентификатора группы (простая адресация);
- **fast failover** – выполняется первое действующее (рабочее) подмножество в группе.

Такой подход позволяет усовершенствовать механизмы форвардинга и реализовать много дополнительных опций. Кроме указанных выше таблиц существуют еще таблицы метрик (Meter table), которые позволяют реализовать примитивные настройки качества обслуживания (QoS), например ограничение полосы пропускания.

1.6.6. Действия (Actions)

Каждому потоку в таблице ассоциировано одно или несколько действий, которые выполняются, когда поток соответствует записи. Под действием понимается операция, которая изменяет пакет, список действий (action set) и/или процесс обработки конвейером, в зависимости от этого различают несколько типов действий:

- **Output** – передать пакет на указанный порт (физический, логический, зарезервированный);
- **Set-Queue** – задать идентификатор очереди для пакета;
- **Drop** – отбросить пакет, принадлежавший потоку;
- **Group** – обработать пакет в соответствии с указанной группой;
- **Push-Tag/Pop-Tag** – работать с метками (VLAN, MPLS);
- **Set-Field** – изменить соответствующие значения поля заголовка пакета;
- **Change-TTL** – изменить значения поля TTL.

В OpenFlow-коммутаторе действия могут быть представлены не по отдельности, а списком действий. Список действий состоит из набора действий, связанных с пакетом, который хранится, пока идет его обработка таблицами потоков, и выполняется только при выходе пакета из конвейера обработки.

1.6.7. Сообщения протокола

Каждый OpenFlow-пакет начинается с заголовка формата, приведенного на рис. 7.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Version	Type	Length	Transaction ID												

Рис. 7. Заголовок протокола OpenFlow

Компоненты заголовка:

- Version – версия протокола OpenFlow;
- Length – длина сообщения, включая заголовок;
- Transaction ID – идентификатор пакета, используется для сопоставления запроса с ответом;
- Type – тип сообщения, может принимать значения, приведенные в табл. 1.

Сообщения протокола OpenFlow

Таблица 1

№	Тип	Направление передачи
<i>Неизменные сообщения</i>		
1	OFPT_HELLO	Симметричное
2	OFPT_ERROR	Симметричное
3	OFPT_ECHO_REQUEST	Симметричное
4	OFPT_ECHO_REPLY	Симметричное
5	OFPT_EXPERIMENTER	Симметричное
<i>Сообщения конфигурирования OpenFlow-коммутатора</i>		
6	OFPT_FEATURES_REQUEST	Контроллер ↔ Коммутатор
7	OFPT_FEATURES_REPLY	Контроллер ↔ Коммутатор
8	OFPT_GET_CONFIG_REQUEST	Контроллер ↔ Коммутатор
9	OFPT_GET_CONFIG_REPLY	Контроллер ↔ Коммутатор
10	OFPT_SET_CONFIG	Контроллер ↔ Коммутатор
<i>Асинхронные сообщения</i>		
11	OFPT_PACKET_IN	Асинхронное
12	OFPT_FLOW_REMOVED	Асинхронное
13	OFPT_PORT_STATUS	Асинхронное
<i>Сообщения-команды контроллера</i>		
14	OFPT_PACKET_OUT	Контроллер ↔ Коммутатор
15	OFPT_FLOW_MOD	Контроллер ↔ Коммутатор
16	OFPT_GROUP_MOD	Контроллер ↔ Коммутатор
17	OFPT_PORT_MOD	Контроллер ↔ Коммутатор
18	OFPT_TABLE_MOD	Контроллер ↔ Коммутатор
<i>Составные сообщения</i>		
19	OFPT_MULTIPART_REQUEST	Контроллер ↔ Коммутатор
20	OFPT_MULTIPART_REPLY	Контроллер ↔ Коммутатор
<i>Сообщения Barrier</i>		
21	OFPT_BARRIER_REQUEST	Контроллер ↔ Коммутатор
22	OFPT_BARRIER_REPLY	Контроллер ↔ Коммутатор
<i>Сообщения конфигурации очередей</i>		
23	OFPT_QUEUE_GET_CONFIG_REQUEST	Контроллер ↔ Коммутатор
24	OFPT_QUEUE_GET_CONFIG_REPLY	Контроллер ↔ Коммутатор
<i>Асинхронные сообщения конфигурации</i>		
25	OFPT_GET_ASYNC_REQUEST	Контроллер ↔ Коммутатор
26	OFPT_GET_ASYNC_REPLY	Контроллер ↔ Коммутатор
27	OFPT_SET_ASYNC	Контроллер ↔ Коммутатор
<i>Сообщения конфигурации таблицы метрик</i>		
28	OFPT_METER_MOD	Контроллер ↔ Коммутатор

1.7. Алгоритм работы ПКС-сети

Далее подробно рассмотрим этапы работы программно-конфигурируемой сети.

Предварительно рассмотрим порядок подключения OpenFlow-коммутатора к контроллеру:

1) при подключении OpenFlow-коммутатора к контроллеру стороны обмениваются сообщением OFPT_HELLO, состоящим из заголовка и набора полей, содержащих версию поддерживаемого протокола и данные, информирующие о начале сеанса связи;

2) после установления сессии контроллер посылает сообщение OFPT_FEATURES_REQUEST для идентификации OpenFlow-коммутатора и считывания его возможностей;

3) коммутатор отвечает сообщением OFPT_FEATURES_REPLY, в котором содержатся поля: однозначная идентификация маршрута (datapath_id), максимальное число буферизованных пакетов (n_buffers) в сообщении (packet-in) при отправке контроллеру, число поддерживаемых таблиц потоков (n_tables), тип соединения (основное или вспомогательное) (auxiliary_id) и комбинация флагов;

4) OpenFlow-коммутатор информирует контроллер об изменении состояния портов (например: «добавлены», «удалены», «конфигурированы») сообщением OFPT_PORT_STATUS;

5) контроллер инициирует запрос OFPMP_PORT_DESC в виде составного сообщения OFPT_MULTIPART_REQUEST на получение описания всех портов сети, которые поддерживают протокол OpenFlow;

6) OpenFlow-коммутатор реагирует ответом в виде одного или нескольких сообщений OFPT_MULTIPART_REPLY с параметрами описания портов OFPMP_PORT_DESC;

7) контроллер конфигурирует OpenFlow-коммутатор командой OFPT_SET_CONFIG или OFPT_GET_CONFIG_REQUEST (очереди, фрагментация, потеря пакетов и т. д.);

8) контроллер посылает запрос OFPT_BARRIER_REQUEST на получение информации о состоянии всех выполняемых операций и настройках на OpenFlow-коммутаторе в данный момент времени;

9) OpenFlow-коммутатор завершает обработку всех запросов, полученных до OFPT_BARRIER_REQUEST, и отправляет ответ OFPT_BARRIER_REPLY («срез» о текущем состоянии OpenFlow-коммутатора);

10) OpenFlow-коммутатор реагирует на запрос конфигурации сообщением OFPT_GET_CONFIG_REPLY;

11) контроллер инициирует запрос OFPMP_DESC в виде составного сообщения на получение описания производителя коммутатора, версии аппаратного обеспечения, серийного номера и др., далее получает ответ;

12) контроллер инициирует запрос OFPMP_TABLE_FEATURES в виде составного сообщения на получение информации об используемых потоковых таблицах (тип, размер, порядок использования) и получает ответ;

13) контроллер инициирует запрос OFPT_ROLE_REQUEST на изменение правил, далее получает ответ;

14) контроллер инициирует запрос OFPT_FLOW_MOD на конфигурацию таблиц потоков;

15) контроллер инициирует запрос OFPT_GROUP_MOD на конфигурацию таблицы групп потоков;

16) контроллер посылает пакет OFPT_PACKET_OUT с широковещательным ARP-запросом для получения MAC-адресов всех подключенных устройств;

17) контроллер получает пакет OFPT_PACKET_IN с ARP-ответом;

18) для проверки соединения контроллер периодически посылает на OpenFlow-коммутатор запросы OFPT_ECHO_REQUEST (содержат только заголовок), а в ответ получает OFPT_ECHO_REPLY. Если запрос соответствует ответу, то соединение находится в рабочем состоянии.

Описанные выше этапы подключения OpenFlow-коммутатора к контроллеру изобразим в виде диаграммы MSC (Message Sequence Chart) (рис. 8).

После завершения подключения всех OpenFlow-коммутаторов к контроллеру сеть находится в режиме ожидания и готова к работе.

Рассмотрим порядок обработки пакета, поступившего с хоста, подключенного к OpenFlow-коммутатору:

1) пакет поступает на вход одного из буферов накопителей OpenFlow-коммутатора;

2) дождавшись своей очереди, пакет начинает обрабатываться: очищается список действий, инициализируются данные о работе конвейера, из пакета извлекаются данные заголовка;

3) конвейерная обработка начинается с нулевой таблицы потоков. Ведется поиск приоритетной записи – анализ на совпадение полей заголовка с классификаторами в таблице потоков (используется алгоритм TCAM). Если найдено несколько соответствующих записей, то выбирается запись с наибольшим приоритетом. Далее выполняются указанные инструкции, обновляются показатели счетчиков и таймеров;

4) инструкции в записи могут явно направить пакет в другую таблицу потоков, где такой же процесс обработки пакета повторяется снова;

5) если при просмотре записи не происходит перенаправления пакета в другую таблицу, то конвейерная обработка для данного пакета завершается и пакет обрабатывается в соответствии с накопившимся списком действий в процессе обработки конвейером;

6) если OpenFlow-коммутатор получил пакет, который не соответствует ни одной записи в таблице потоков, в инструкции указан непосредственный вывод на контроллер или неверно указано значение поля TTL, то копия инкапсулируется и по защищенному каналу посылается на контроллер;

7) контроллер обрабатывает пакет и указывает, из какого порта OpenFlow-коммутатора его необходимо отправить или просто отбросить;

8) после применения всех действий пакет отправляется из указанного порта.

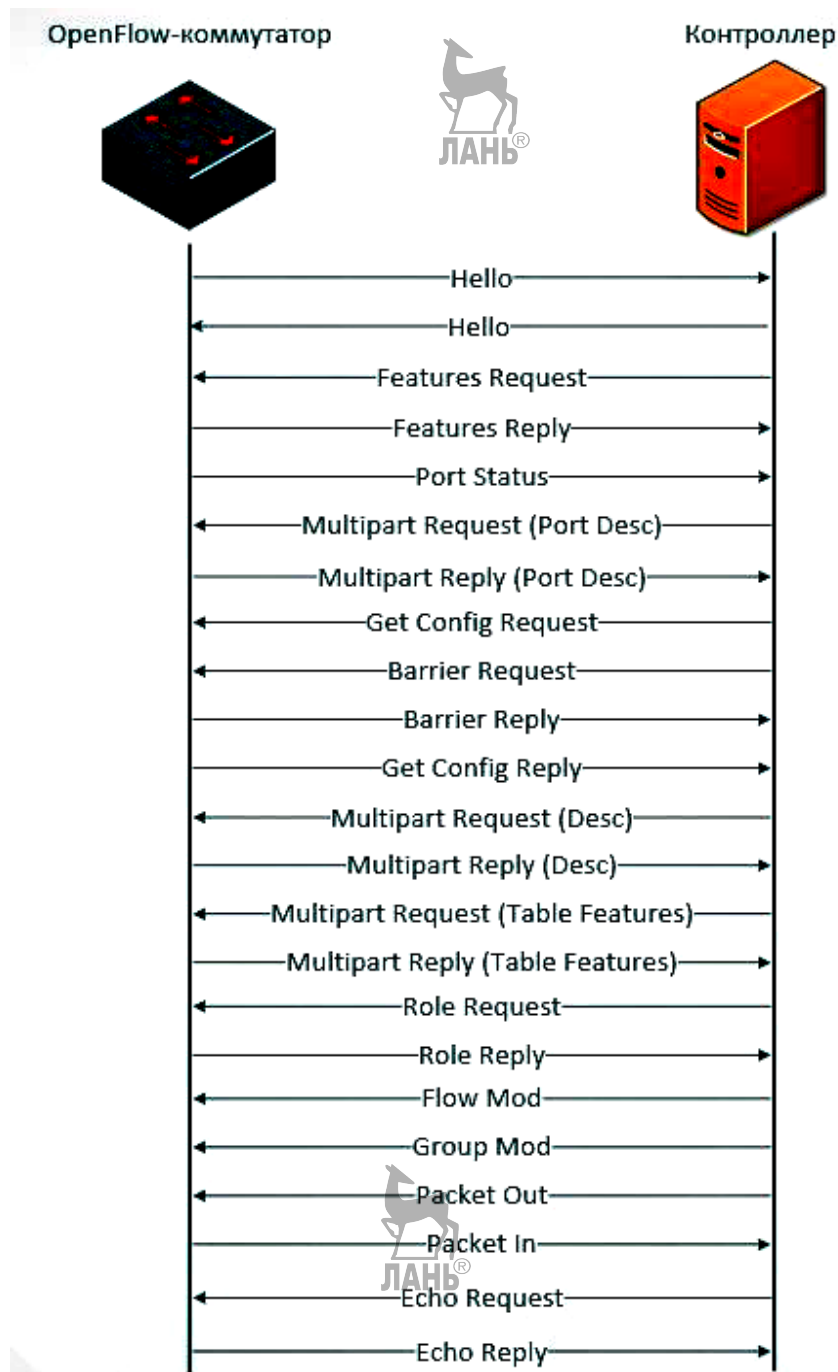


Рис. 8. MSC-диаграмма обмена сообщениями при подключении OpenFlow-коммутатора

Этапы обработки пакета, отправленного с хоста, подключенного к OpenFlow-коммутатору, в виде цифр изображены на схеме рис. 9.

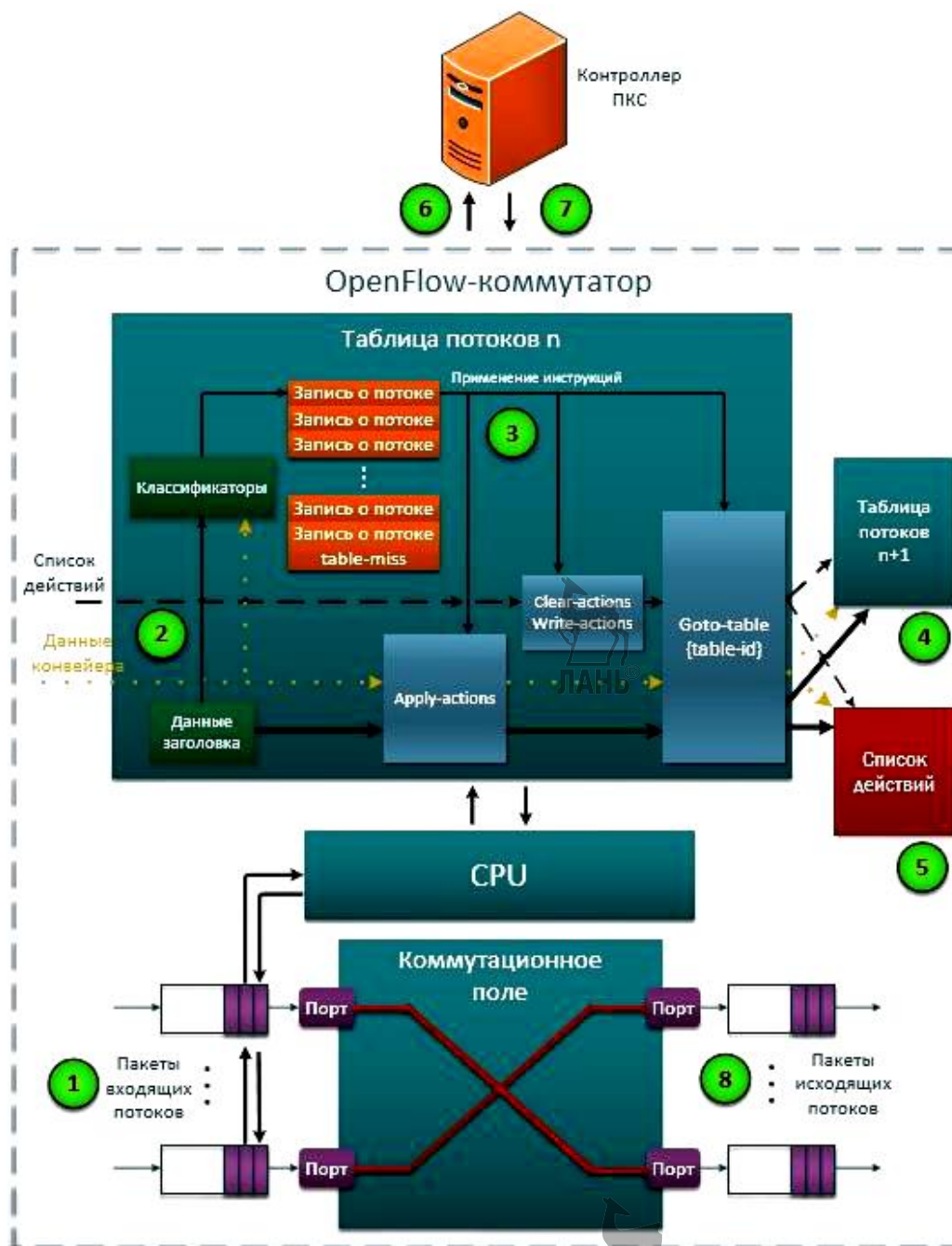


Рис. 9. Алгоритм работы OpenFlow-коммутатора

1.8. Классификаторы протокола OpenFlow

В разд. 1.6.5 частично были описаны поля и структура классификаторов, которые являются одной из нескольких составляющих таблиц потоков. С выходом новых версий протокола OpenFlow строение классификаторов постепенно усложнялось. Начиная с версии протокола 1.3, каждый

классификатор может состоять из заголовка и иметь от нуля до некоторого количества полей соответствия классификаторов, закодированных при помощи формата OXM TLV. Типы и классы, из которых состоят классификаторы протокола OpenFlow, приведены в виде схемы на рис. 10.

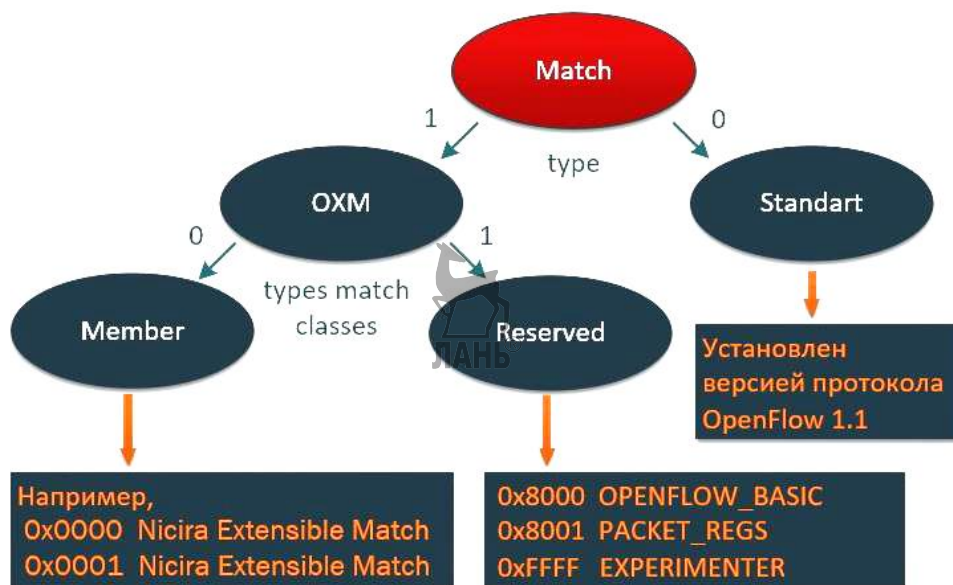


Рис. 10. Типы классификаторов протокола OpenFlow

1.8.1. Заголовок классификаторов

Поля заголовка:

- **Туре** – определены два типа классификаторов. Первый тип – стандартный (значение поля «тип» равняется нулю), определены версией протокола OpenFlow 1.1 и считаются устаревшими, но базовыми для всех OpenFlow-коммутаторов. Второй тип – расширяемые классификаторы OpenFlow Extensible Match (OXM);

- **Length** – динамически может изменяться в указанных пределах, зависит от количества полей классификаторов;
- **OXM fields** – поля классификаторов;
- **Pad** – нулевой байт.

1.8.2. OXM fields

Все поля соответствия классификатора описываются с использованием динамически расширяемого формата OXM. В совокупности (вместе) они образуют компактный формат type-length-value (TLV). Классификаторы OXM TLV могут быть от 5 до 259 (включительно) байт длиной.

Первые 4 байта каждого классификатора OXM TLV – заголовок (рис. 11):

- 1) **OXM class** – спецификация OpenFlow разделяет два типа OXM классов:

– ONF reserved – классы, описываемые спецификацией и зарезервированные для последующих стандартизаций;

– ONF member – классы, используемые членами консорциума ONF. Позволяют членам консорциума использовать свои классификаторы и уникально идентифицироваться, благодаря им (напр. Nicira).

2) OXM field – поле используется для определения конкретного поля соответствия в каждом типе класса (табл. 2 и 3);

3) OXM hasmask – поле может принимать значения 0 или 1. Определяет наличие в поле данных OXM битовой маски;

4) OXM length – длина полезных данных OXM TLV.

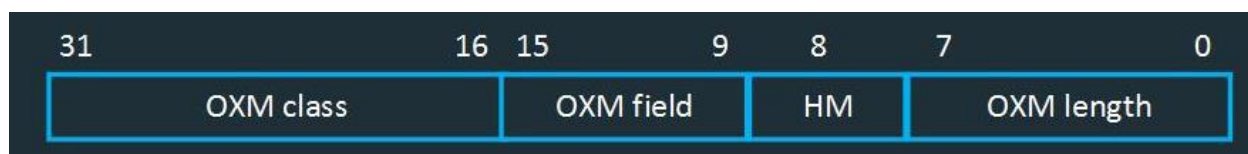


Рис. 11. Заголовок классификатора OXM TLV

1.8.3. OXM classes – reserved

Тип OXM-классов – Member – не описывается спецификациями и не требует разбора в рамках данной работы. Рассмотрим классы Reserved на примере протокола OpenFlow версии 1.5.1.

В спецификации протокола определены три класса в данном типе:

- experimenter – класс используется для расширения стандартного набора полей соответствия в OpenFlow-коммутаторе;
- register – класс используется для хранения временных значений и информации, связанной с пакетом в процессе обработки конвейером;
- OpenFlow basic – основной класс классификаторов, который содержит базовый набор полей соответствия, однако не все из этого набора полей должны обязательно поддерживаться OpenFlow-коммутатором, но каждое обязательное match-поле должно поддерживаться не менее чем одной таблицей потоков. Кроме базовых полей соответствия основного класса таблицы потоков могут содержать match-поля (в необязательном порядке) классов experimenter и register. Поля соответствия класса OpenFlow basic разделяют на два типа. К первому типу относятся поля, которые сравниваются с данными, извлеченными из заголовков пакетов (header match fields). Все они имеют разные размеры, обязательность/необязательность к реализации, маску и описание (табл. 2).



Таблица 2

Поля соответствия класса OpenFlow basic (header match fields)

№	Math-поле	Размер, бит	Маска	Описание
1	OFPXMT_OFB_ETH_DST	48	Да	MAC-адрес получателя
2	OFPXMT_OFB_ETH_SRC	48	Да	MAC-адрес отправителя
3	OFPXMT_OFB_TYPE	16	Нет	Ethernet type
4	OFPXMT_OFB_VLAN_VID	12+1	Да	VLAN-ID по стандарту 802.1Q
5	OFPXMT_OFB_VLAN_PCP	3	Нет	VLAN-PCP по стандарту 802.1Q
6	OFPXMT_OFB_IP_DSCP	6	Нет	Diff Serv Code Point (DSCP)
7	OFPXMT_OFB_IP_ECN	2	Нет	ECN бит в заголовке IP
8	OFPXMT_OFB_IP_PROTO	8	Нет	IPv4- или IPv6-номер протокола
9	OFPXMT_OFB_IPV4_SRC	32	Да	IPv4-адрес отправителя
10	OFPXMT_OFB_IPV4_DST	32	Да	IPv4-адрес получателя
11	OFPXMT_OFB_TCP_SRC	16	Нет	TCP-порт отправителя
12	OFPXMT_OFB_TCP_DST	16	Нет	TCP-порт получателя
13	OFPXMT_OFB_TCP_FLAGS	12	Нет	TCP-флаги
14	OFPXMT_OFB_UDP_SRC	16	Нет	UDP-порт отправителя
15	OFPXMT_OFB_UDP_DST	16	Нет	UDP-порт получателя
16	OFPXMT_OFB_SCTP_SRC	16	Нет	SCTP-порт отправителя
17	OFPXMT_OFB_SCTP_DST	16	Нет	SCTP-порт получателя
18	OFPXMT_OFB_ICMPV4_TYPE	8	Нет	ICMP-тип
19	OFPXMT_OFB_ICMPV4_CODE	8	Нет	ICMP-код
20	OFPXMT_OFB_ARP_OP	16	Нет	ARP-код отправителя/ получателя
21	OFPXMT_OFB_ARP_SPA	32	Да	IPv4-адрес отправителя в payload протокола ARP
22	OFPXMT_OFB_ARP_TPA	32	Да	IPv4-адрес получателя в payload протокола ARP
23	OFPXMT_OFB_ARP_SHA	48	Да	MAC-адрес отправителя в payload протокола ARP
24	OFPXMT_OFB_ARP_THA	48	Да	MAC-адрес получателя в payload протокола ARP
25	OFPXMT_OFB_IPV6_SRC	128	Да	IPv6-адрес отправителя
26	OFPXMT_OFB_IPV6_DST	128	Да	IPv6-адрес получателя
27	OFPXMT_OFB_IPV6_FLABEL	20	Да	IPv6-метка потока
28	OFPXMT_OFB_ICMPV6_TYPE	8	Нет	ICMPv6-тип
29	OFPXMT_OFB_ICMPV6_CODE	8	Нет	ICMPv6-код
30	OFPXMT_OFB_IPV6_ND_TAR-GET	128	Нет	IPv6, сообщение NDM, адрес получателя

№	Math-поле	Размер, бит	Маска	Описание
31	OFPXMT_OFB_IPV6_ND_SLL	48	Нет	IPv6, сообщение NDM, source link-layer address option
32	OFPXMT_OFB_IPV6_ND_TLL	48	Нет	IPv6, сообщение NDM, target link-layer address option
33	OFPXMT_OFB_MPLS_LABEL	20	Нет	MPLS-метка
34	OFPXMT_OFB_MPLS_TC	3	Нет	MPLS-класс трафика
35	OFPXMT_OFB_MPLS_BOS	1	Нет	MPLS флаг «дно стека»
36	OFPXMT_OFB_PBB_ISID	24	Да	Технология PBB, идентификатор сервиса I-SID
37	OFPXMT_OFB_IPV6_EXTHDR	9	Да	IPv6 Extension Header pseudo-field
38	OFPXMT_OFB_PBB_UCA	1	Нет	Технология PBB, поле UCA

Второй тип полей соответствия содержит поля, которые используются в процессе обработки конвейером (pipeline match fields) и никаким образом не взаимодействуют с заголовками пакета (табл. 3).

Таблица 3

Поля соответствия класса OpenFlow basic (pipeline match fields)

№	Math-поле	Размер, бит	Маска	Описание
1	OFPXMT_OFB_IN_PORT	32	Нет	Входящий порт. Номер логического или физического входящего порта, начиная с единицы
2	OFPXMT_OFB_IN_PHY_PORT	32	Нет	Физический порт. Соответствие физического и логического порта, когда сообщение приходит на логический порт
3	OFPXMT_OFB_METADATA	64	Да	Используется для передачи информации между таблицами
4	OFPXMT_OFB_TUNNEL_ID	64	Да	Метаданные, ассоциированные с логическим портом
5	OFPXMT_OFB_ACTSET_OUT-PUT	32	Нет	Output port from action set Metadata
6	OFPXMT_OFB_PACKET_TYPE	16+16	Нет	Packet type – canonical header type of outermost header

Однако, как и каждая технология, SDN имеет свои недостатки.

Во-первых, каждому коммутатору, несмотря на использование OpenFlow, приходится обрабатывать большие заголовки пакетов, в поисках со-

ответствия их полей в записях своих flow-таблиц. Разумеется, плюсом здесь является то, что стандарт OpenFlow рассматривает потоки пакетов, а не отдельные пакеты, соответственно обрабатывается только заголовок первого пакета потока.

Во-вторых, коммутаторам необходимо хранить большие объемы данных в flow-таблицах, что приводит к необходимости поддержания достаточно большого объема памяти и вынуждает использовать более мощные и соответственно достаточно дорогие платы. В-третьих, реализация всей функциональности OpenFlow на одном устройстве может подвергнуть сеть сбоям в результате перегрузки контроллера. Например, даже легальные запросы могут перегрузить контроллер и устроить сбой сети, если коммутатор настроен так, чтобы при получении TCP-пакета с флагом SYN каждый раз отправлять контроллеру.

Контрольные вопросы

1. На каком интерфейсе (южном или северном) используется протокол OpenFlow?
2. Для чего в SDN сетях необходим контроллер?
3. Какой протокол транспортного уровня используется для OpenFlow?
4. Какие основные функции выполняет коммутатор SDN?
5. Каковы основные подходы к формированию архитектуры SDN? Опишите их.
6. Какие международные организации участвуют в стандартизации SDN?



2. ЛАБОРАТОРНЫЕ РАБОТЫ

Используемые инструменты

Эмулятор сети Mininet

Сетевой эмулятор Mininet позволяет быстро развернуть сеть, состоящую из различного числа виртуальных хостов, коммутаторов, контроллеров и каналов между ними.

Данное программное обеспечение использует технологии виртуализации, встроенные в ядро ОС Linux. Это позволяет запускать на виртуальных хостах стандартные сетевые утилиты Linux. Эмулируемые с помощью Mininet коммутаторы имеют поддержку OpenFlow, что позволяет использовать данный продукт для исследования и обучения принципам построения ПКС. При этом вся сеть (до 4096 устройств) может быть запущена на ноутбуке или компьютере без подключения физических устройств, которые зачастую имеют немалую стоимость.

Большая часть кода Mininet написана с использованием языка Python. Топологии эмулируемых сетей, а также параметры сетевых устройств также могут быть описаны на языке Python. Кроме того, существуют встроенные шаблоны топологий, например, линейная топология или топология «дерево». Также для управления эмулируемой сетью существует встроенный интерпретатор команд.

Более подробная информация, а также программное обеспечение для скачивания содержатся на официальном сайте проекта <http://mininet.org/>.

Контроллер ПКС Floodlight

Floodlight OpenFlow SDN Controller – это OpenFlow-контроллер корпоративного класса, распространяющийся под лицензией Apache.

Floodlight имеет модульную структуру. Таким образом, перед запуском в специальный конфигурационный файл добавляются названия сервисов, которые будут использоваться.

Floodlight поддерживает следующие сервисы:

- Forwarding – реактивный режим обработки и пересылки пакетов;
- Static Flow Entry Pusher – приложение для добавления записей потока (классификатора и действий) в выбранный коммутатор;
- Firewall – модуль применения ACL-правил для ограничения трафика на основании классификаторов;
- Port Down Reconciliation – модуль для согласования маршрутов движения трафика в случае, если произойдет отказ в работе одного из портов;
- Learning Switch – режим поведения обычного L2-коммутатора;

-
- Hub – приложение, которое всегда рассылает входящие пакеты на все активные порты;
 - Virtual Network Filter – модуль для простой изоляции сетей на основе MAC-адресов;
 - Load Balancer – простой модуль балансировки нагрузки для потоков tcp, ping, udp.

Более подробную информацию о контроллере, а также информацию по установке можно найти на сайте разработчика по адресу: <http://www.projectfloodlight.org/>.

Анализатор сетевого трафика Wireshark

Частой задачей лабораторных работ будет исследование процесса обмена трафиком между узлами сети. Для этих целей может быть использован анализатор сетевых пакетов Wireshark. Начиная с версии 1.12.0, Wireshark позволяет перехватывать и распознавать сообщения протокола OpenFlow. Также сетевой анализатор может определить структуру сообщения и передаваемые в нем данные.



Лабораторная работа 1

ЗАПУСК МОДЕЛЬНОЙ СЕТИ SDN

Цель работы: научиться создавать и запускать модельную сеть SDN, запускать контроллер модельной сети и подключать к нему виртуальные коммутаторы.

Порядок работы

1. Запустите эмулятор Mininet.

Рассмотрим запуск модельной сети SDN на примере системы Ubuntu Linux, на которую уже установлены Mininet и контроллер Floodlight. Готовая виртуальная машина с установленным необходимым набором ПО может быть скачана с сайта проекта Floodlight. Если вы хотите самостоятельно установить и настроить эмулятор Mininet и контроллер Floodlight, обратитесь к документации соответствующих проектов.

Для запуска эмулятора Mininet достаточно выполнить следующую команду:

```
floodlight@floodlight:~$ sudo mn
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet>
```

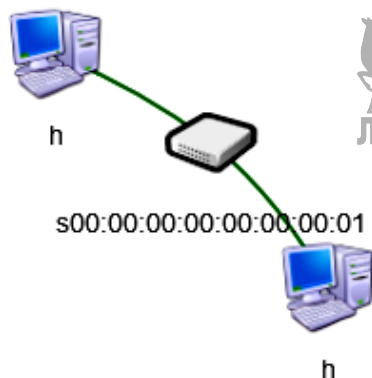


Рис. 12. Схема простейшей сети Mininet

В этом случае будет создана простейшая топология, состоящая из двух хостов, коммутатора и контроллера, как показано на рис. 12, и запущена консоль управления Mininet.

2. Проверьте работу виртуальной сети.

Чтобы проверить связь между хостами созданной сети выполните команду *pingall*. В результате выполнения команды каждый хост выполнит *ping* со всеми остальными хостами. В нашем случае это будет выглядеть следующим образом:

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
```

3. Завершите работу эмулятора.

Для завершения работы Mininet выполните команду *exit*:

```
mininet> exit
*** Stopping 1 controllers
c0
*** Stopping 2 links
..
*** Stopping 1 switches
s1
*** Stopping 2 hosts
h1 h2
*** Done
completed in 180.283 seconds
```



4. Создайте топологию сети.

Mininet позволяет создавать топологии сетей с помощью различных шаблонов. Например, шаблон *tree* позволяет создавать древовидную топологию, а *linear* – линейную, где каждый коммутатор последовательно соединен с каждым:

```
floodlight@floodlight:~$ sudo mn --topo linear,4 --switch \
ovsk --ipbase=10.0.0.0/8
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1 s2 s3 s4
*** Adding links:
(h1, s1) (h2, s2) (h3, s3) (h4, s4) (s2, s1) (s3, s2) (s4, s3)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
c0
*** Starting 4 switches
s1 s2 s3 s4 ...
*** Starting CLI:
mininet>
```



5. Запустите программу на хостах.

Mininet позволяет запускать на узлах простые сетевые приложения Linux. Выполните команду `ping` с первого хоста на третий, для этого введите следующую команду:

```
mininet> h1 ping h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=8.94 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=0.277 ms
^C
--- 10.0.0.3 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 0.277/4.610/8.944/4.334 ms
```

6. Запустите контроллер FloodLight.

Контроллер FloodLight может быть запущен с помощью следующей команды:

```
java -jar target/floodlight.jar &
```

После того как контроллер был запущен, можно получить доступ к графическому веб-интерфейсу управления контроллером. Чтобы подключиться к веб-интерфейсу контроллера, введите в адресной строке веб-браузера следующий адрес:

<http://<ip-адрес>:8080/ui/index.html>,

где *ip-адрес* – адрес контроллера FloodLight.

Внешний вид веб-интерфейса представлен на рис. 13.

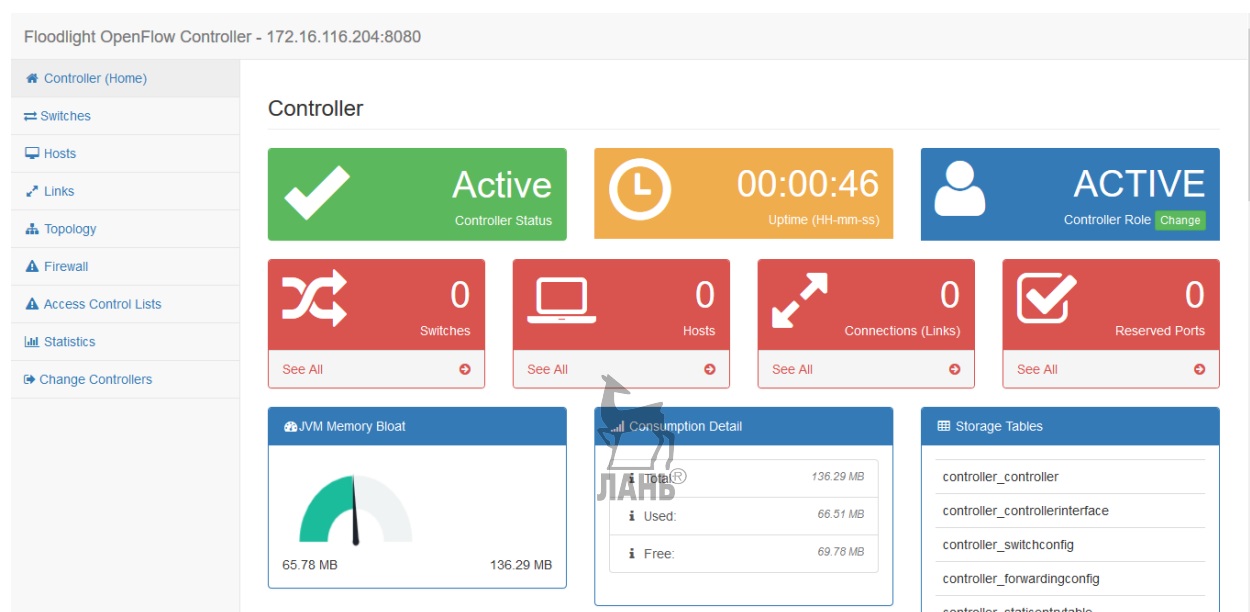


Рис. 13. Графический интерфейс контроллера FloodLight

7. Подключите виртуальную сеть к контроллеру.

Для подключения коммутаторов виртуальной сети Mininet к контроллеру необходимо в параметрах запуска указать `--controller=remote`. Выполните следующую команду:

```
sudo mn --topo linear,4 --switch ovsk --ipbase=10.0.0.0/8 \
--controller=remote,ip=<ip-адрес>,port=6653
```

где *ip-адрес* – адрес контроллера FloodLight; 6653 – порт, используемый по умолчанию для протокола OpenFlow.

Перейдите на страницу «Topology» и убедитесь, что к контроллеру была подключена виртуальная сеть из четырех коммутаторов, как показано на рис. 14.

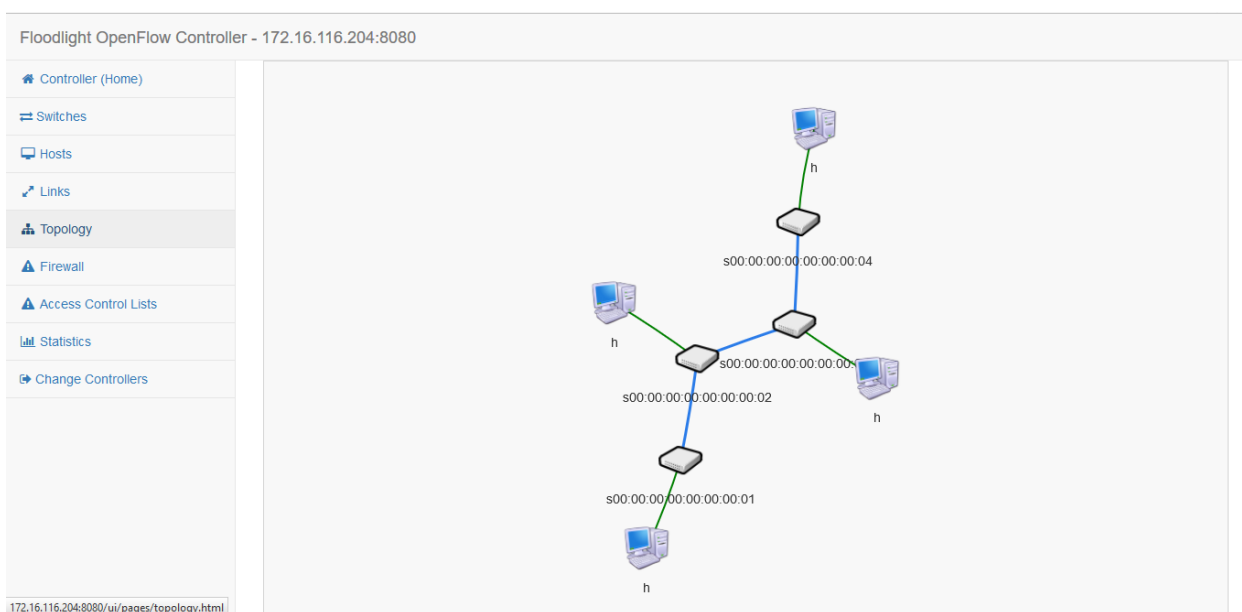


Рис. 14. Топология сети, подключенной к контроллеру



Лабораторная работа 2

АНАЛИЗ ПАКЕТОВ OpenFlow

Цель работы: научиться перехватывать пакеты, передающиеся при взаимодействии OpenFlow-коммутатора и контроллера, а также проводить анализ перехваченного трафика.

Порядок работы

1. Запустите сетевой анализатор Wireshark на узле, на котором установлен контроллер.
2. В параметрах запуска анализатора выберите интерфейс «ану», в этом случае трафик будет перехватываться со всех пакетов. Запустите процесс сбора трафика с помощью кнопки «Start».
3. В качестве фильтра пакетов укажите «openflow_v4», что соответствует пакетам протокола OpenFlow версии 1.3.
4. Запустите контроллер FloodLight.
5. Запустите эмулятор сети Mininet, создав в нем линейную топологию из четырех коммутаторов, аналогично тому, как это было выполнено в лабораторной работе 1.
6. Запустите ping между первым и четвертым хостами сети:
`mininet> h1 ping h4`
7. С помощью сетевого анализатора Wireshark проанализируйте обмен пакетами между коммутаторами и контроллером.

Ответьте на вопросы.

- Какой порт протокола TCP используется для передачи сообщений OpenFlow?
- Какая версия протокола OpenFlow используется при взаимодействии коммутатора и контроллера?
- Какие сообщения протокола OpenFlow используются при взаимодействии между коммутатором и контроллером?
- Какое действие указано в сообщении OFPT_PACKET_OUT для потока данных?



Лабораторная работа 3

СОЗДАНИЕ ПРАВИЛ ОБРАБОТКИ ПОТОКА НА КОНТРОЛЛЕРЕ SDN

Цель работы: научиться создавать правила обработки различных потоков трафика с помощью контроллера SDN.

Порядок работы

1. Запустите сетевой анализатор Wireshark на узле, на котором установлен контроллер. Запустите сбор трафика со всех интерфейсов контроллера и отфильтруйте по протоколу OpenFlow версии 1.3.
2. Запустите контроллер FloodLight.
3. Запустите эмулятор сети Mininet, создав в нем линейную топологию из четырех коммутаторов.
4. С помощью команды *pingall* в консоли Mininet проверьте связь между узлами виртуальной сети.
5. В графическом интерфейсе контроллера FloodLight перейдите на страницу «Access Control List». Создайте новое правило обработки трафика с помощью кнопки «Add New».

В появившемся окне укажите следующие параметры:

протокол – ICMP;

исходящий адрес – 10.0.0.1/32;

адрес назначения – 10.0.0.4/32;

действие – DENY.

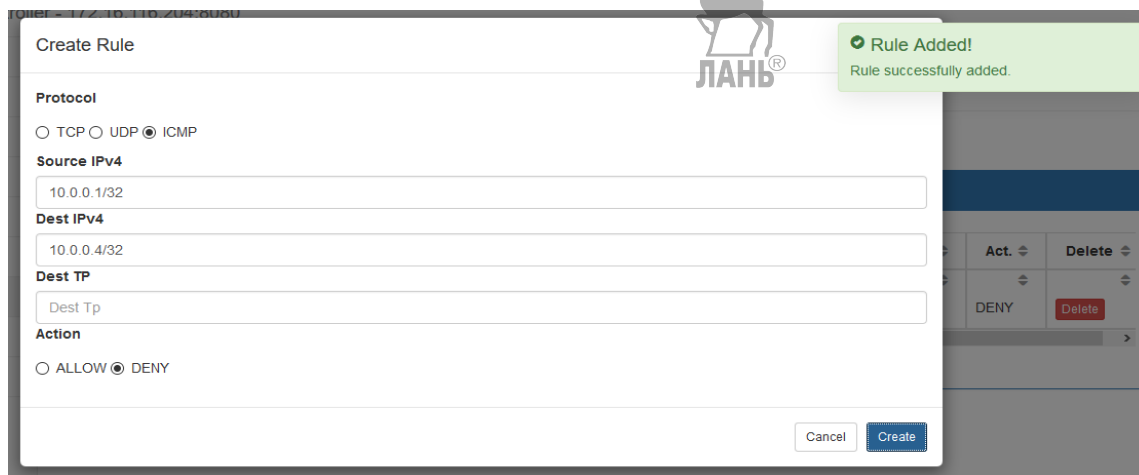


Рис. 15. Создание правила обработки трафика

6. Повторно выполните *pingall*.
7. Проанализируйте полученный результат.

Ответьте на вопросы.

- Как изменилась связность между узлами?
- Как изменился обмен сообщениями OpenFlow?

Создание топологий в mininet

В этом упражнении вы научитесь создавать пользовательские топологии с помощью Mininet Python API и узнаете, как можно индивидуально настраивать такие параметры, как пропускная способность, задержка, потери и размер очереди для различных каналов в топологии. Вы также узнаете, как проводить тестирование производительности этих пользовательских топологий с помощью `ping` и `iperf`.

После обзора вам будет предложено создать и отправить собственную топологию, основанную на наиболее распространенной трехуровневой архитектуре ЦОД, т. е. ядре, агрегации и уровне доступа.

Обзор

Сеть, которую вы будете использовать в этом упражнении, включает хосты и коммутаторы, соединенные в линейную топологию, как показано на рисунке ниже.

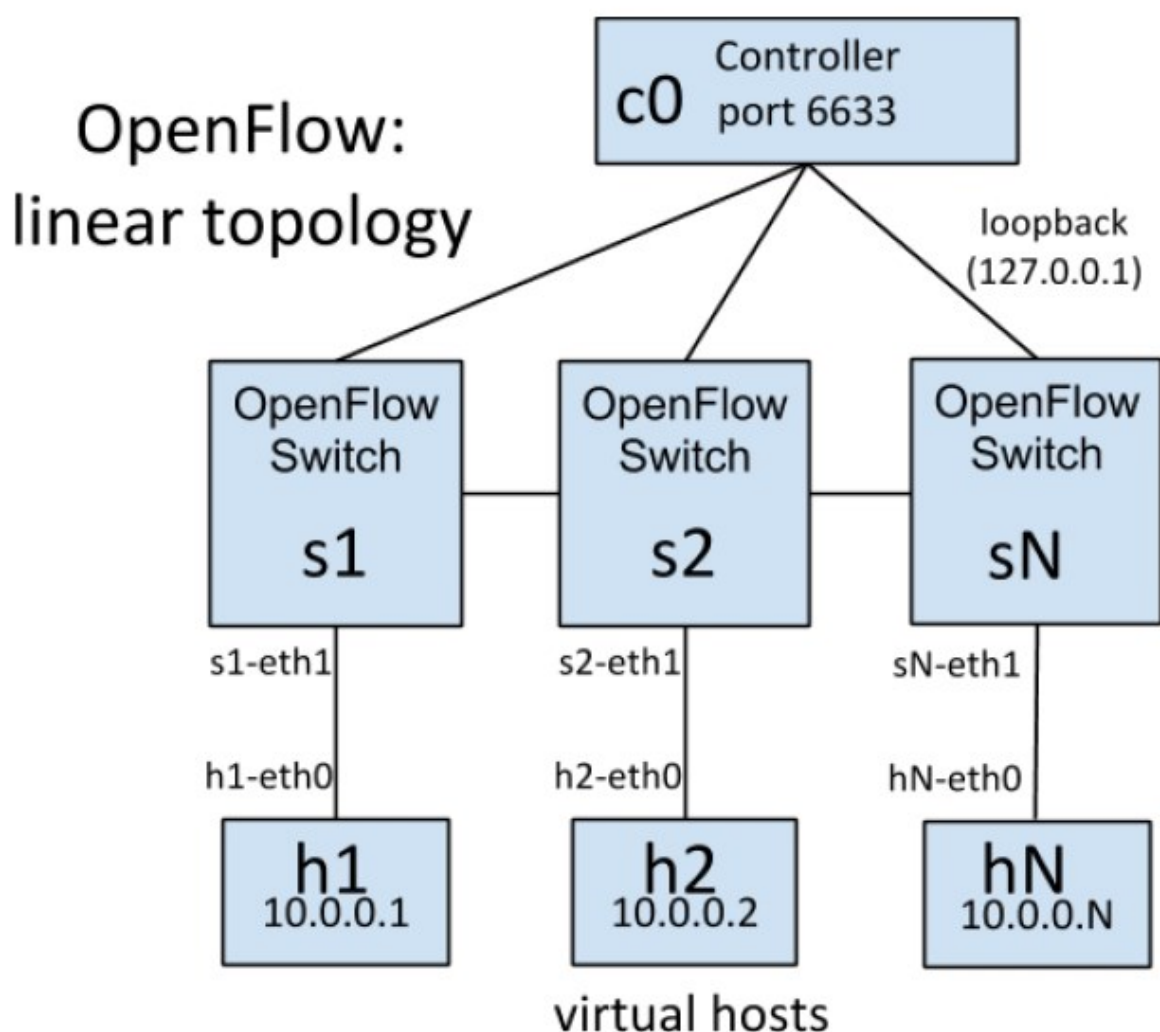


Рис. 1 – Простая топология

Создание топологии

Mininet поддерживает *параметризованные топологии*. С помощью нескольких строк кода на Python вы можете создать гибкую топологию, которая может быть настроена на основе переданных в нее параметров и повторно использована для нескольких экспериментов.

Например, вот простая топология сети (на рис. 1), состоящая из определенного количества хостов ($h1 - hN$), подключенных к отдельным коммутаторам ($s1 - sN$):

```
#!/usr/bin/python

from mininet.topo import Topo
from mininet.net import Mininet
from mininet.util import irange, dumpNodeConnections
from mininet.log import setLogLevel

class LinearTopo(Topo):
    "Linear topology of k switches, with one host per switch."

    def __init__(self, k=2, **opts):
        """Init.

        k: number of switches (and hosts)
        hconf: host configuration options
        lconf: link configuration options"""

        super(LinearTopo, self).__init__(**opts)

        self.k = k

        lastSwitch = None
        for i in irange(1, k):
            host = self.addHost('h%s' % i)
            switch = self.addSwitch('s%s' % i)
            self.addLink( host, switch)
            if lastSwitch:
                self.addLink( switch, lastSwitch)
            lastSwitch = switch

def simpleTest():
    "Create and test a simple network"
    topo = LinearTopo(k=4)
    net = Mininet(topo)
    net.start()
    print "Dumping host connections"
    dumpNodeConnections(net.hosts)
    print "Testing network connectivity"
    net.pingAll()
    net.stop()

if __name__ == '__main__':
    # Tell mininet to print useful information
    setLogLevel('info')

    simpleTest()
```

Важные классы, методы, функции и переменные в приведенном выше коде включают:

- `Topo`: базовый класс для топологий Mininet
- `addSwitch()`: добавляет коммутатор в топологию и возвращает его имя
- `addHost()`: добавляет хост в топологию и возвращает его имя.
- `addLink()`: добавляет двунаправленную связь в топологию (и возвращает ключ связи, но это не важно). Связи в Mininet являются двунаправленными, если не указано иное.
- `Mininet`: основной класс для создания и управления сетью
- `start()`: запускает вашу сеть
- `pingAll()`: тестирует соединение, пытаясь заставить все узлы пинговать друг друга
- `stop()`: останавливает работу вашей сети
- `net.hosts`: все хосты в сети
- `dumpNodeConnections()`: дамп соединений с/из набора узлов.
- `setLogLevel('info' | 'debug' | 'output')`: установка уровня вывода Mininet по умолчанию; рекомендуется использовать 'info', так как он предоставляет полезную информацию.

Настройка параметров производительности

В дополнение к базовому поведенческому сетевому взаимодействию Mininet предоставляет функции ограничения производительности и изоляции с помощью классов `CPULimitedHost` и `TCLink`.

Существует множество способов использования этих классов, но один из простых - указать их в качестве хоста по умолчанию и связать классы/конструкторы с `Mininet()`, а затем указать соответствующие параметры в топологии.

```
#!/usr/bin/python

from mininet.topo import Topo
from mininet.net import Mininet
from mininet.node import CPULimitedHost
from mininet.link import TCLink
from mininet.util import irange, dumpNodeConnections
from mininet.log import setLogLevel

class LinearTopo(Topo):
    "Linear topology of k switches, with one host per switch."

    def __init__(self, k=2, **opts):
        """Init.
        k: number of switches (and hosts)
        hconf: host configuration options
        lconf: link configuration options"""

        super(LinearTopo, self).__init__(**opts)

        self.k = k

        lastSwitch = None
        for i in irange(1, k):
            host = self.addHost('h%s' % i, cpu=.5/k)
            switch = self.addSwitch('s%s' % i)
```

```

        # 10 Mbps, 5ms delay, 1% loss, 1000 packet queue
        self.addLink( host, switch, bw=10, delay='5ms', loss=1,
max_queue_size=1000, use_htb=True)
        if lastSwitch:
            self.addLink(switch, lastSwitch, bw=10, delay='5ms', loss=1,
max_queue_size=1000, use_htb=True)
            lastSwitch = switch

def perfTest():
    "Create network and run simple performance test"
    topo = LinearTopo(k=4)
    net = Mininet(topo=topo,
                  host=CPULimitedHost, link=TCLink)
    net.start()
    print "Dumping host connections"
    dumpNodeConnections(net.hosts)
    print "Testing network connectivity"
    net.pingAll()
    print "Testing bandwidth between h1 and h4"
    h1, h4 = net.get('h1', 'h4')
    net.iperf((h1, h4))
    net.stop()

if __name__ == '__main__':
    setLogLevel('info')

    perfTest()

```

Некоторые важные методы и параметры:

`self.addHost(name, cpu=f)`: Это позволяет указать долю общих системных ресурсов процессора, которая будет выделена виртуальному хосту.

`self.addLink( node1, node2, bw=10, delay='5ms', max_queue_size=1000, loss=1, use_htb=True)`: добавляет двунаправленный канал с характеристиками пропускной способности, задержки и потерь, с максимальным размером очереди в 1000 пакетов, используя ограничитель скорости Hierarchical Token Bucket и эмулятор задержки/потерь `netem`. Параметр `bw` выражается в виде числа в Мб/с; задержка выражается в виде строки с единицами измерения на месте (например, '5ms', '100us', '1s'); потери выражаются в процентах (от 0 до 100); и `max_queue_size` выражается в пакетах.

Вам может пригодиться создание словаря Python, чтобы, например, легко передавать одни и те же параметры в несколько вызовов методов:

```

linkopts = dict(bw=10, delay='5ms', loss=1, max_queue_size=1000,
use_htb=True)
'''
alternately: linkopts = {'bw':10, 'delay':'5ms', 'loss':1,
'max_queue_size':1000, 'use_htb':True}
'''

self.addLink(node1, node2, **linkopts)

```

Запуск в Mininet

Чтобы запустить созданную выше пользовательскую топологию, следуйте приведенным ниже инструкциям:

- Создайте сценарий LinearTopo.py на вашей VM Mininet и скопируйте в него содержимое LinearTopology (без настроек производительности), перечисленное выше.

- Сделайте скрипт исполняемым

```
$ chmod u+x LinearTopo.py
```

- Выполните скрипт

```
$ sudo ./LinearTopo.py
```

Выход

```

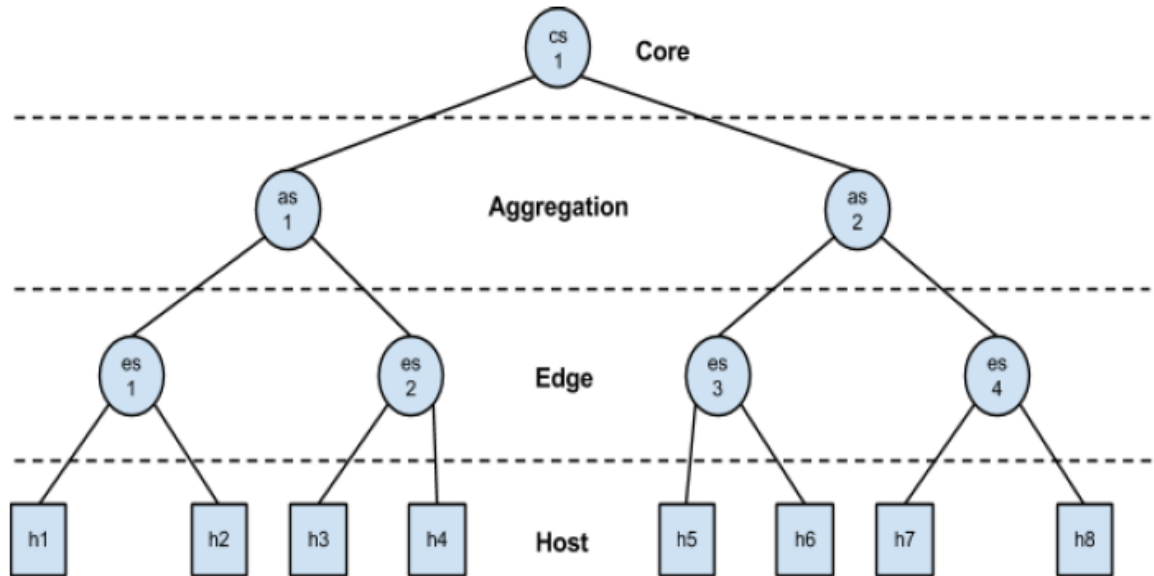
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1 s2 s3 s4
*** Adding links:
(h1, s1) (h2, s2) (h3, s3) (h4, s4) (s1, s2) (s2, s3) (s3, s4)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
*** Starting 4 switches
s1 s2 s3 s4
Dumping host connections
h1 h1-eth0:s1-eth1
h2 h2-eth0:s2-eth1
h3 h3-eth0:s3-eth1
h4 h4-eth0:s4-eth1
Testing network connectivity
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (0/12 lost)
*** Stopping 4 hosts
h1 h2 h3 h4
*** Stopping 4 switches
s1 ...s2 ....s3 ....s4 ...
*** Stopping 1 controllers
c0
*** Done

```

Задание 1

Сети центров обработки данных обычно имеют древовидную топологию. Конечные узлы подключаются к коммутаторам верхней части стойки, которые образуют листья (края) дерева; один или несколько основных коммутаторов образуют корень; и один или несколько уровней агрегирующих коммутаторов образуют середину дерева. В базовой древовидной топологии каждый коммутатор (кроме основного) имеет один родительский коммутатор. Повышения отказоустойчивости или увеличения пропускной способности между стойками могут быть добавлены дополнительные коммутаторы и линии связи для построения более сложных топологий дерева (например, толстого дерева).

В этом задании ваша задача - создать простую древовидную топологию. Предполагается, что каждый уровень, т. е. ядро, агрегация, граница и хост, состоит из одного слоя коммутаторов/хостов с настраиваемым значением fanout (k). Например, простая древовидная сеть с одним слоем на каждом уровне и fanout, равным 2, выглядит следующим образом:



Задание 2

Создать кольцевидную топологию с настраиваемым значением fanout (k). Также ваша логика должна поддерживать установку, по крайней мере, параметров bw и delay для каждого соединения.

